

ТЗ по проекту «Путь клиента»

История изменений технического задания

Что было изменено	Кто изменил	Когда

1. Цели проекта и критерии их достижения

Предоставить инструмент для маркетологов и менеджеров для создания автоматических сценариев взаимодействия и коммуникации с покупателями, позволяющий:

- Настраивать логику с учетом различных условий и фильтров
- Определять последовательность действий
- Автоматически реагировать на изменения и действия клиентов
- Адаптировать сценарии под разные сегменты клиентов на основе их данных и поведения

Критерий достижения: можно настроить следующие сценарии:

1. Возвращение клиента
Покупатель не осуществлял покупки более 30 дней, начислить бонусы, подсчитать результаты на предмет покупок в течении 10 дней
2. Брошенная корзина
Покупатель не завершил покупку в течении 2 дней, напомнить рассылкой, подсчитать результаты на предмет завершения покупки в течение 10 дней

2. Потенциальный круг пользователей

Потенциальные клиенты

- **Маркетологи и менеджеры по лояльности:** В компаниях, использующих Saby для CRM (например, ритейл, услуги). Они создают сценарии, анализируют результаты.
- **Владельцы малого/среднего бизнеса:** Для простых кампаний, без IT-специалистов, благодаря шаблонам. Акцент на предустановках.

<https://i.saby.ru/page/component-statistics-feature-card/682>

Фишки		Найти	
Ресторан Presto			
		11 нн'25	
		11 нн'25	
		Активности Аккаунты Активности Аккаунты Купили % Вес	
		393 435 756 7 262 825 597 055 7 995 9 223 9.31	
Продажи		344 553 514 6 846 765 429 969 7 477 8 767 5.96 100	
Учет и госсистемы		9 965 356 5 588 13 820 930 6 149 7 992 9.47 100	
Маркетинг		4 290 749 1 874 7 191 826 2 146 8 337 4.07 100	
Бонусная программа		7 616 619 7 010 660 7 992 8.26 100	
Скидки		152 638 698 261 751 600 7 992 2.73 100	
Промокоды		14 840 140 157 553 173 7 992 0.64 100	
Электронные карты		189 912 735 287 659 796 7 992 4.57 100	
Реферальная программа		346 58 275 63 7 992 0.79 100	
База клиентов		1 934 542 548 2 299 704 642 7 992 8.03 100	
Рассылки		1 447 203 586 2 364 968 678 7 992 3.41 100	
Разовые		2 039 289 3 881 312 7 992 3.9 100	
Повторяющиеся		3 784 406 6 145 406 7 992 5.08 100	
СМС		6 686 433 15 780 477 7 992 5.97 100	
Мессенджеры		9 704 406 8 145 406 7 992 5.08 100	

Используем для основы клиентов использующих Рассылки в продуктах престо ~670 аккаунтов.

За первый месяц внедрения - 5 аккаунтов.

За последующие 6 мес - 100 аккаунтов настроят сценарии лояльности.

3. Концептуальное описание решения

Все решение строится на базе скриптов, маршрутов и системы триггеров.

Основные работы

- [Прикладники] [Триггерные события и действия](#) для использования в платформе маршрутов
- [Лояльность] Реестр и карточка сценариев лояльности, на базе компонентов и методов БЛ маршрутов и скриптов
 - [Платформа Маршрутов] Заведение нового типа маршрутов
- [Платформа Маршрутов] Механики помещения клиента на маршрут, на основе
 - Триггерных событий (например *Покупка, Выдача карты, Брошенная корзина* и т.д.)
 - Разово либо периодически запускаемой выборки
- + Ограничения попадания на маршрут, режимы: "временя тишины", "попадание на маршрут не чаще раз в N времени"
- [Платформа Маршрутов] Авто-проходчик маршрутов. Клиент попадая на маршрут движется платформой маршрута до конечного этапа с возможным "засыпанием" на этапах "Ожидания"
 - Добавление этапа "Ожидани" на маршрут. Реализуется по проекту [Таймер внутри маршрута](#)
 - Добавление этапа "Действие" на маршрут
- [Платформа Маршрутов] Карточка прохода, для возможности просмотра прохода Клиента в рамках конкретного маршрута, с возможностью выйти на весь перечень маршрутов в котором участвует Клиент
- [Платформа Маршрутов] Возможность сохранения мета-информации о событиях (стартового либо на этапах ожидания) прохода Клиента по маршруту (например о покупке) для возможности отображения этой мета информации (например о покупке) на Карточке прохода. *Агрегация и статистика по продажам произошедшим в рамках маршрута, в текущем проекте не реализуется!*



Для прикладников методы и компоненты под-системы маршрутов (Окладников Д., Чусовитин А.) и скриптов (Гаттаулин Э.) это все [Платформа Маршрутов]

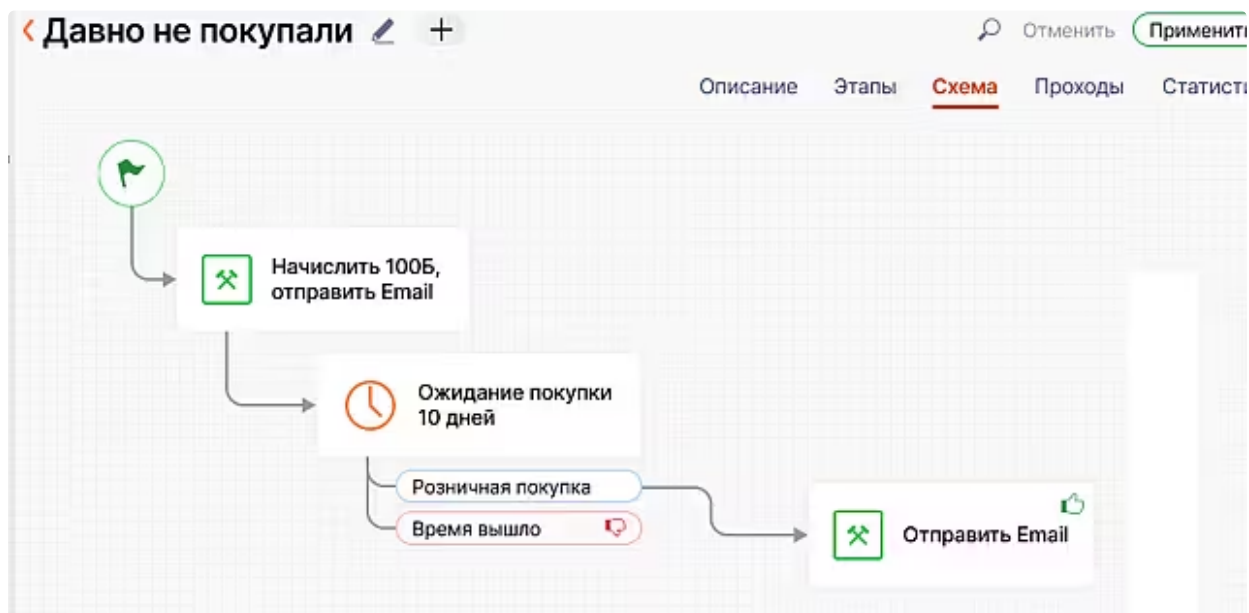
Примеры сценариев, закрывающиеся по проекту

▼ Реактивация неактивных клиентов (давно не покупали)

Цель: Вернуть клиентов, которые перестали покупать.

Что добъёмся:

- **Боремся с оттоком:** Системно "оживляем" базу клиентов, которая со временем теряет активность.



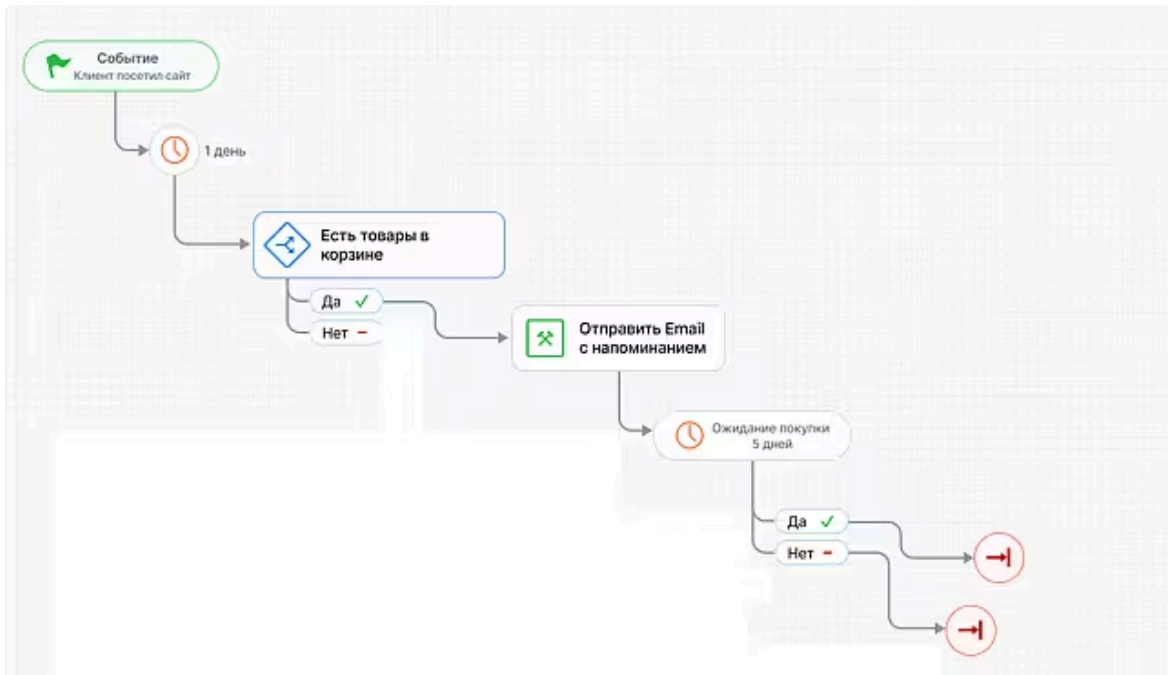
▼ Брошенная корзина

- **Цель:**

Вернуть в магазин клиентов, которые проявили интерес (добавили товары в корзину или просто смотрели товары), но не завершили покупку.

- **Что добъёмся:**

- **Снижаем процент брошенных корзин:** Напоминаем о некупленных товарах, помогая клиенту принять окончательное решение.

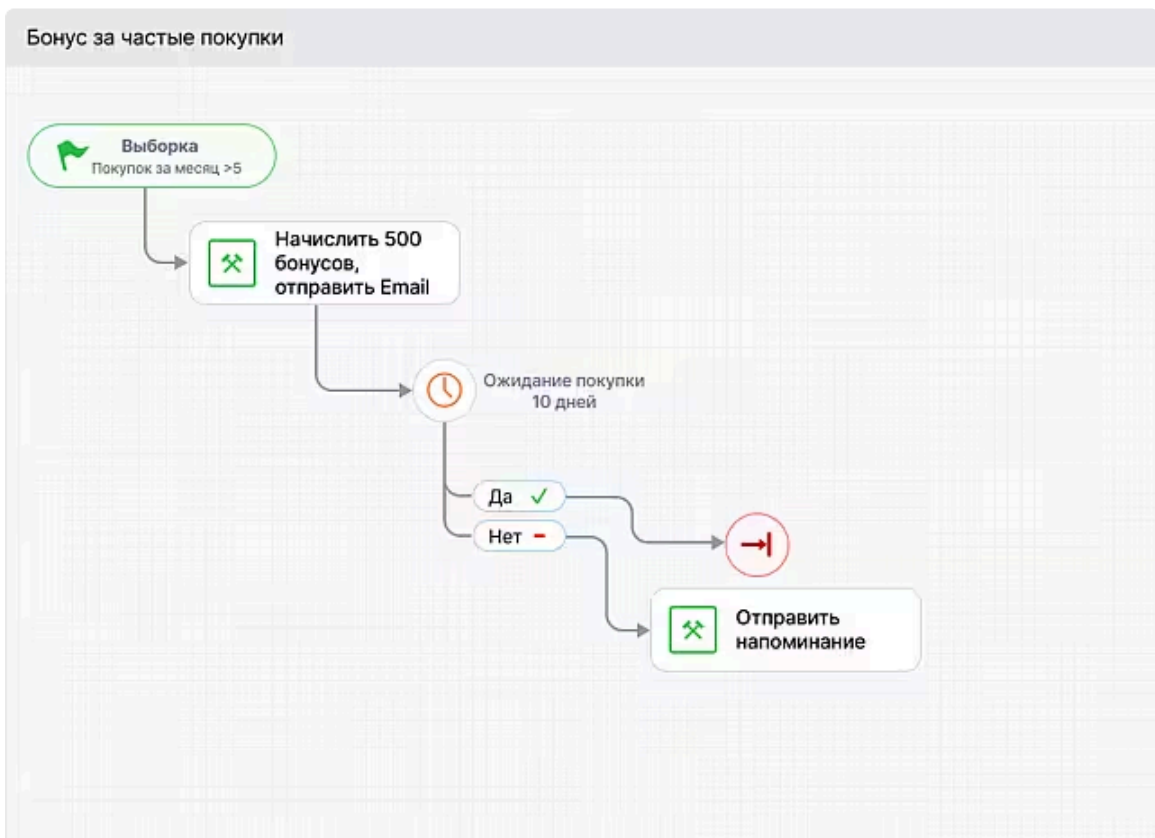


▼ Бонус за частые покупки

- **Цель:** Поощрить и удержать самых активных (лояльных) клиентов.

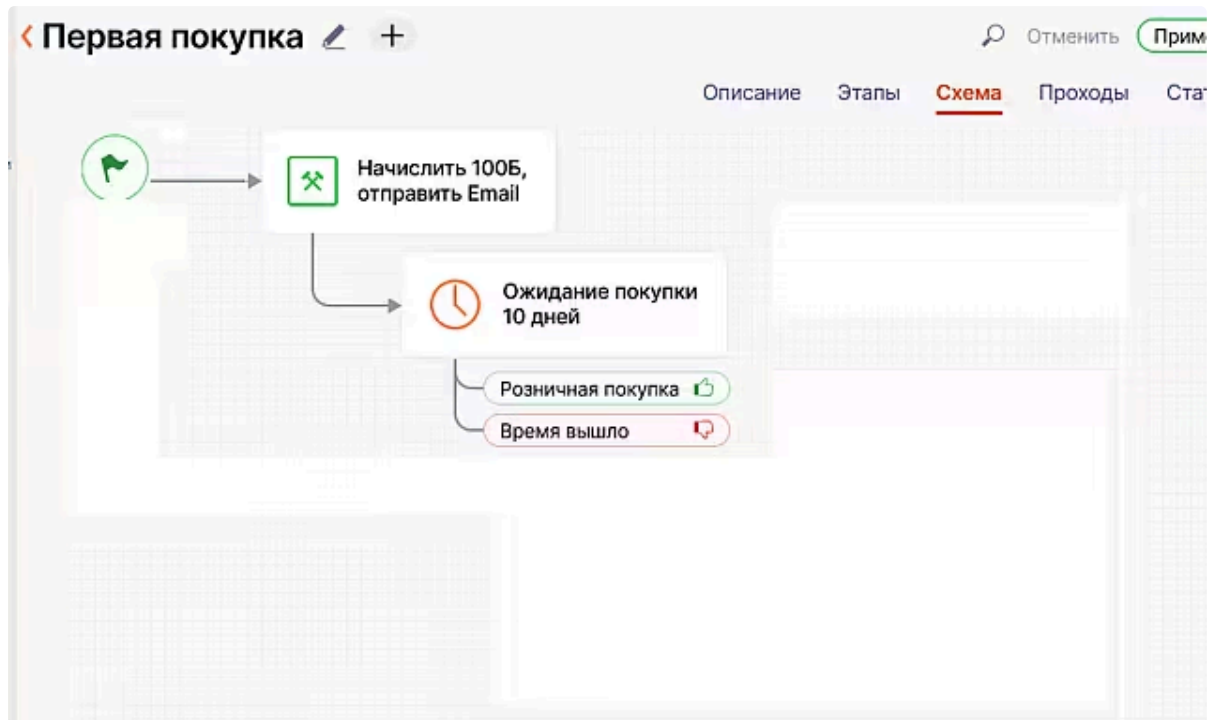
- **Что добъёмся:**

- **Увеличиваем частоту покупок:** Вознаграждаем клиентов, которые покупают чаще 5 раз в месяц, крупным бонусом, подкрепляя их привычку покупать.



▼ Мотивация на вторую покупку

- **Цель:** Сформировать у нового клиента положительное впечатление о бренде с самого начала и мотивировать его на вторую покупку.
- **Что добьемся:**
 - **"Разогреваем" новичков:** Сразу после первой покупки начисляем бонусы и отправляем приветственное письмо, повышая лояльность.



Понятийная модель

▼ Схема

Клиент - Покупатель, которого с помощью сценариев побуждают к совершению покупок

Сценарий-Маршрут (Лояльности) - Является **Маршрутом**. Задаёт периодичность и правила отбора попадания клиента на маршрут, схему прохождения клиентов и перечень целей

- **Узел (Этап)** - Определяется выполняемым действием при переходе на этап, а так же возможными переходами (**Ребрами**) на другие этапы

Проход - Совокупность переходов клиента от попадания на маршрута до завершения

- **Событие** (мотивация к переходу) - Определяет движение клиента от одного этапа к другому (по ребру), в рамках прохода маршрута

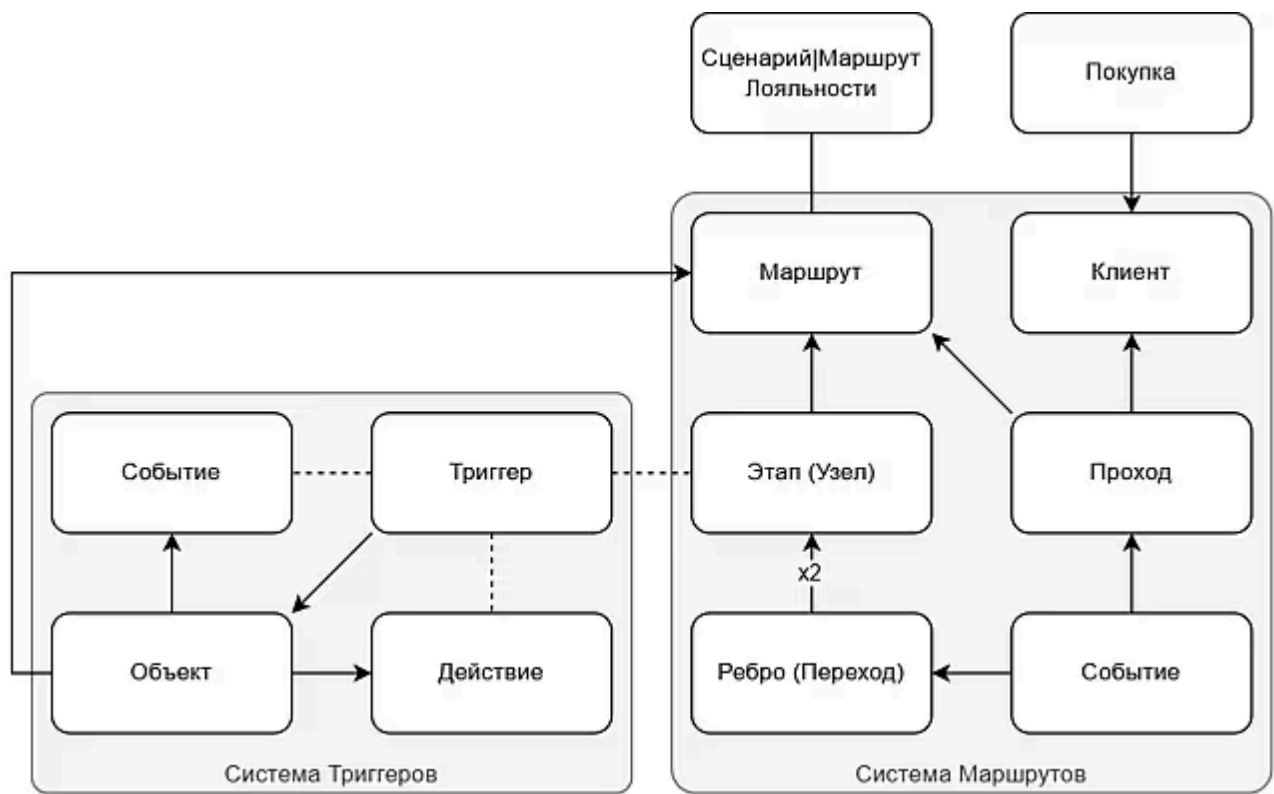
Покупка - Осуществленная покупка клиентом находящимся на маршруте в момент не завершенного прохода

Система триггеров

- **Событие** - Какое либо событие определяемое прикладной областью (например "совершена покупка")
- **Действие** - Какое либо действие определяемое прикладной областью как реакция на событие задающееся в Триггере
- **Объект** - Определяет множество триггерных Событий и множество возможных на них Действий

На маршруте стартовому этапу на основе события и этапу "Ожидание", будет соответствовать тех. триггер (создаваемый платформой маршрутов) с соответствующим триггерным событием

На маршруте этапу "Действие" будет соответствовать тех. триггер (создаваемый платформой маршрутов) с заполненным блоком действия, но не с заполненным блоком событие т.к. данный триггер будет "поджигаться" платформой маршрутов в авто-проходчике



Подсистемы и зоны ответственности

Решение состоит из следующих подсистем

- **Лояльность (ТД)**, предоставляет и отвечает за
 - Страницу сценариев лояльности, конфигурирование и компоновку реестра и карточки маршрутов
 - Триггерные события и действия лояльности, и настройку маппинга данных (в системе триггеров) между ними
- **Платформа маршрутов (ТД)**, предоставляет и отвечает за
 - Авто-проходчик клиента по маршруту
 - Имплементация этапа "Действия", "Ожидания" в авто-проходчике
 - Добавление клиентов на маршрут на основе триггерных событий и выборов
 - Карточка прохода
 - Фиксация мета информации о событиях в проходе для последующего отображения на карточке (для отображения покупки в карточке прохода на маршрутах лояльности)

Скрипты (ТД), предоставляют и отвечает за

- Базовые компоненты реестра, карточки маршрутов и их деталей, включая UI редактор схем
- Карточку этапа Действия и Ожидания
- Блок настройки старта маршрута и режимов добавления клиента на маршрут (по событию и выборке)
- **Триггеры (ТД)**, предоставляют и отвечают за
 - Механизмы платформы триггерных событий, действий и объектов для задействования в маршрутах
 - Возможность указания пользователем параметров действий (например для действия "Начислить бонусы" указать число "100", или для действия "Добавить клиента в папку" указать конкретную папку)
- **Выборки (ТД)**, предоставляют и отвечают за
 - Асинхронный механизм отбора клиентов согласно выборке
 - Механизм ответа на вопрос "попадает Клиент в выборку?" (Для сценария маршрута по событию, но с пост-фильтрацией по выборке)
- **Прочие под-системы**
 - Розница, предоставляет событие "Оплаты продажи"
 - Под-система трекинга система маркетинга + виджеты SabyGet и МП Saby Get, предоставляет событие "Брошенная корзина", "Добавление в корзину", "Просмотр товара"
 - CRM, предоставляет действие Добавление|Удаление Клиента в|из папки

Модель данных

- Данные сценария лояльности, ложатся на модель хранения Маршрута и не предполагают какого либо расширения
- Факт покупки Клиентом в разрезе маршрута, хранится на проходе см. подробнее [ТЗ 6.0.2.3 Детализация о продаже в карточке прохода](#) без возможности агрегации по сумме покупок (но возможность агрегации закладывается на будущее)

Основные механизмы

- [ТЗ 6.0.1.3 Этап "Действие"](#)
- [ТЗ 6.0.1.4 Этап "Ожидание"](#)
- [ТЗ 6.0.2.1 Помещение клиента на маршрут](#)
- [ТЗ 6.0.2.2 Авто-проходчик маршрутов](#)
- [ТЗ 6.0.2.3 Детализация о продаже в карточке прохода](#)

4. Интерфейс

Макет	Состояние макета
Реестр Сценарии лояльности	Согласовано (используются общие элементы реестра скриптов и чат ботов)
Карточка сценария	Согласовано (используются общие элементы реестра скриптов и чат ботов)
Карточка прохода клиента	Устно согласован

[Предыдущая версия макета \(с примерами схем сценариев\)](#)

- [Описание бизнес-процесса;](#)

Экран сценариев (Список)

На примере конфигурации Saby Retail:

Доступ к разделу осуществляется через навигационное меню: **Скидки** -> **Скидки и акции** -> **Сценарии**

При первом входе (когда нет созданных маршрутов) отображается приветственный экран «Персональный подход».

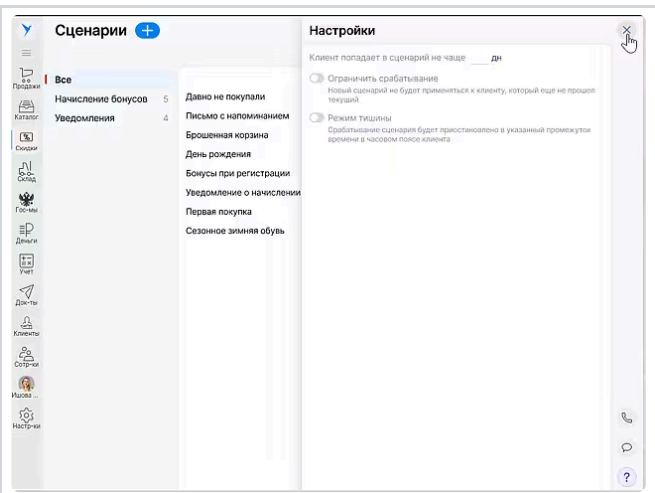
Пользователю предлагается создать сценарий с нуля или использовать один из четырех предустановленных шаблонов для быстрого старта:

- Бонусы при регистрации
- Давно не покупали
- День рождения
- Брошенная корзина

Глобальные настройки (Шестеренка)

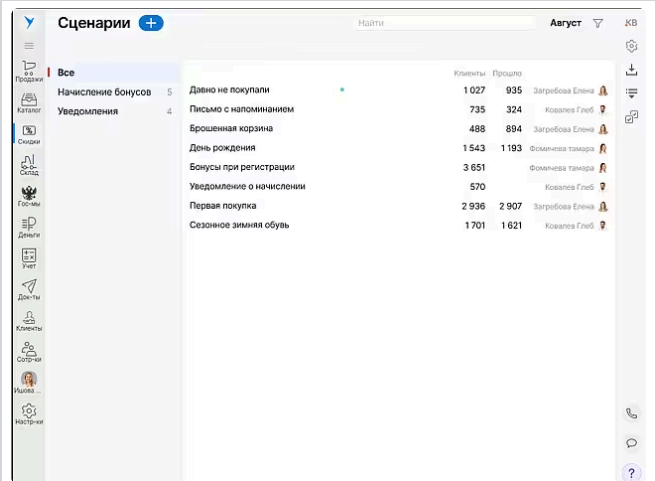
По клику на иконку шестеренки (в правом верхнем углу интерфейса) открывается боковая панель общих настроек, где задаются ограничения для попадания клиентов в сценарии:

- **Частота:** Поле для ввода лимита (например, «Клиент попадает в сценарий не чаще N дн»).
- **Ограничить срабатывание:** Тумблер, запрещающий применять новый сценарий к клиенту, если он еще не завершил текущий проход.
- **Режим тишины:** Тумблер для приостановки срабатывания сценария в заданный промежуток времени (с учетом часового пояса клиента)



Главный экран представляет собой реестр, разделенный на две рабочие зоны:

- **Левая панель (Навигация):** Позволяет фильтровать список по папкам (например, «Все», «Начисление бонусов», «Уведомления»).
- **Правая панель (Список):** Отображает перечень настроенных сценариев.
 - Для каждого пункта выводится: название, счетчик «Клиенты» (попавшие в выборку) и счетчик «Прошло» (завершившие маршрут).
- **Верхняя панель (Тулбар):** Содержит кнопку добавления «+», строку поиска, фильтр по периоду который повлияет на счетчи клиентов в сценариях

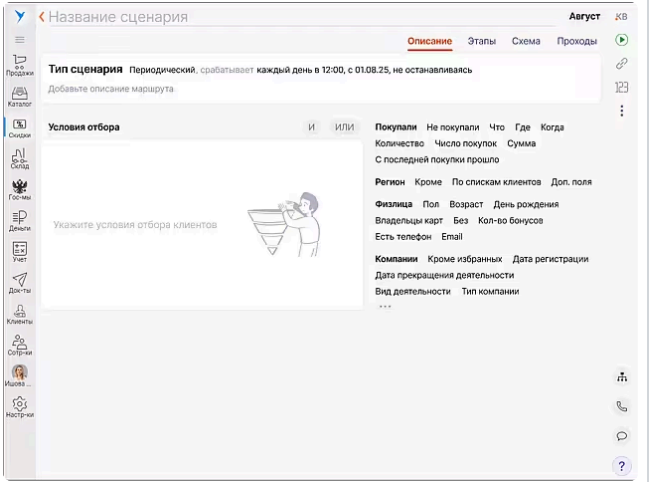
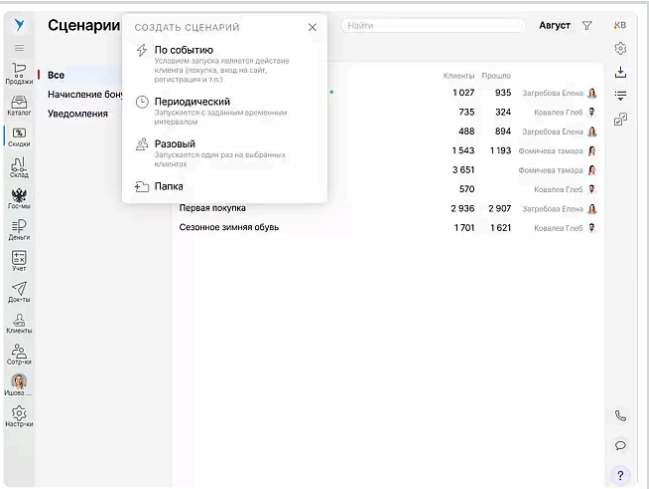


Меню создания сценария (Кнопка «+») При клике на «+» открывается выпадающее меню с выбором типа сценария и возможностью создать новую папку. Типы сценариев:

- **По событию:** Запускается по триггеру (покупка, вход на сайт и т.д.). Наведение на этот пункт открывает вложенное меню с иерархией доступных объектов и событий (Лояльность -> По дисконтной карте; Клиент -> Розничная покупка и т.д.).
- **Периодический:** Запускается с заданным временным интервалом.
- **Разовый:** Единичный запуск по выбранному списку клиентов.

В зависимости от выбранного типа сценария, интерфейс создания адаптируется:

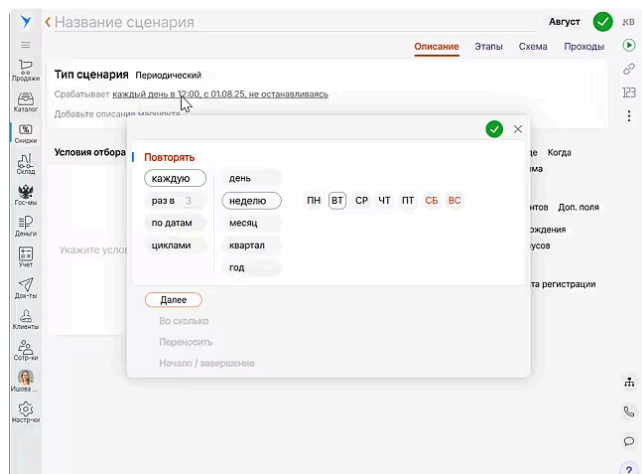
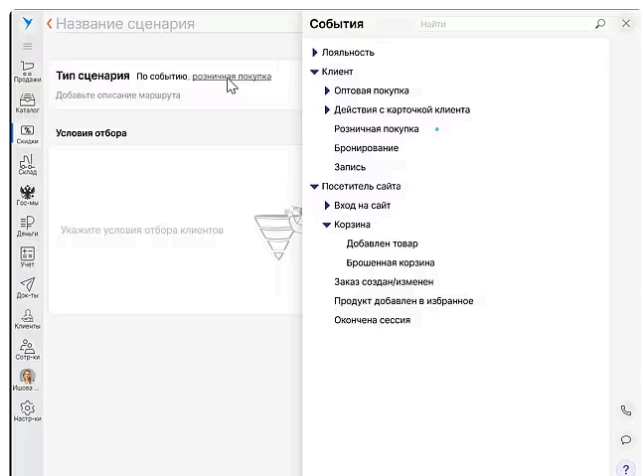
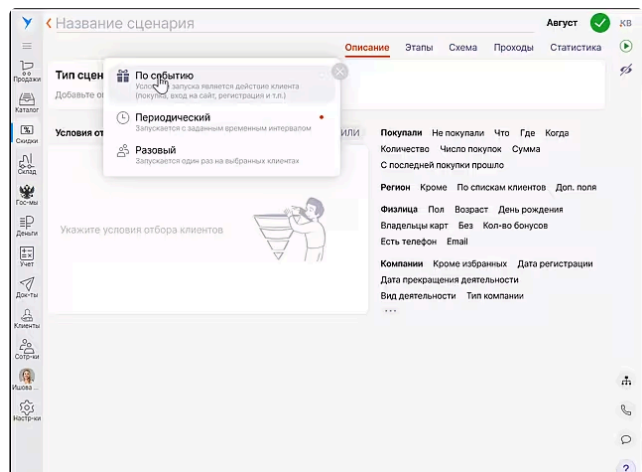
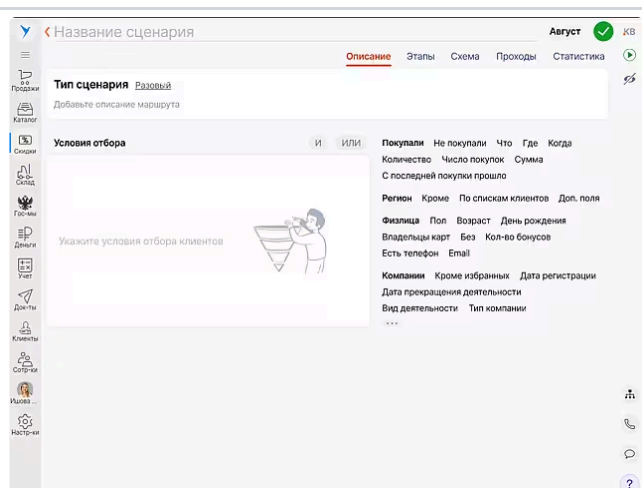
Открывается полноэкранная карточка сценария. В шапке указывается тип сценария и текстовое описание расписания (например, «срабатывает каждый день в 12:00...»). Ниже доступно поле для ввода описания маршрута и блок «Условия отбора» с аналогичным компонентом сегментации клиентов для формирования периодической выборки



Экран сценария

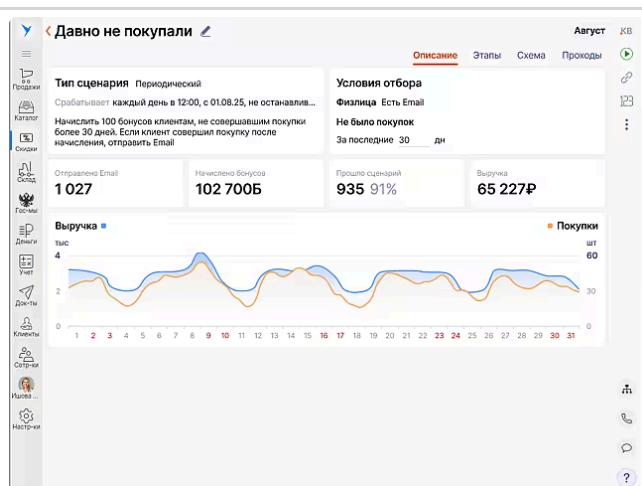
Внутри карточки (до запуска) вкладка «Описание» позволяет гибко настраивать базовые параметры:

- **Смена типа сценария:** При клике на текущий тип сценария открывается выпадающее окно, где можно «на лету» изменить логику (По событию / Периодический / Разовый).
- **Настройка триггеров (По событию):** При клике на выбранное событие (например, «Розничная покупка») справа выезжает панель «События» с иерархическим реестром (Лояльность, Клиент, Посетитель сайта и т.д.), где можно перевыбрать триггер.
- **Настройка расписания (Периодический):** При клике на текстовое описание расписания открывается модальное окно «Повторять». Оно содержит детальный календарь повторений (каждую неделю, раз в месяц, по датам, циклами) и выбор конкретных дней недели.



После сохранения и активации сценария вкладка «Описание» трансформируется в аналитический дашборд:

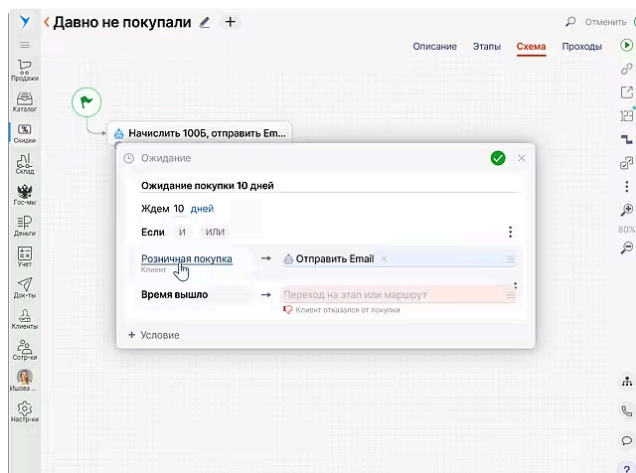
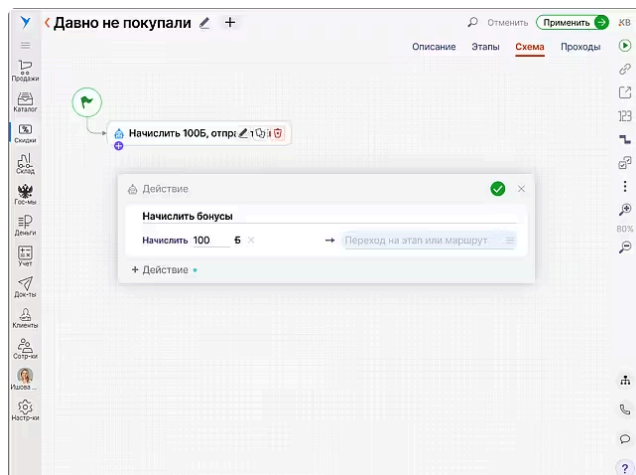
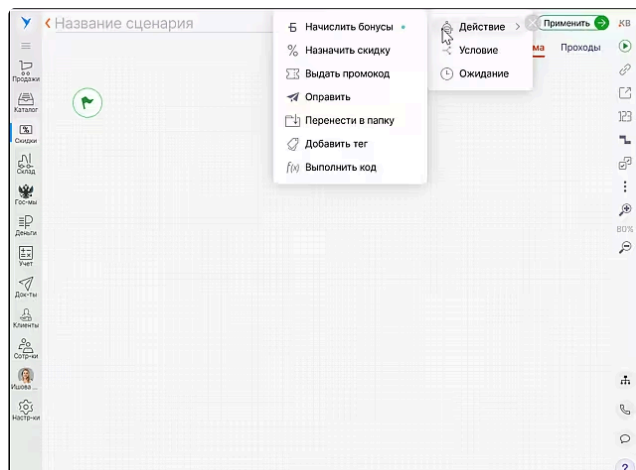
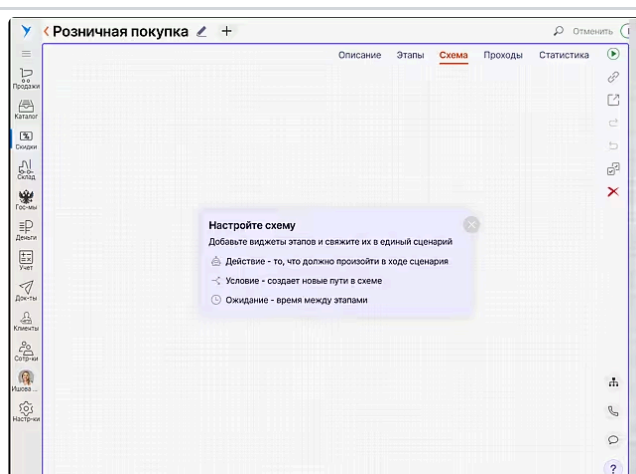
- **Шапка:** Появляются иконки управления — «Карандаш» (переход в режим редактирования) и кнопка Play/Pause для остановки или запуска сценария.
- **Виджеты метрик:** Выводится сводная статистика — Отправлено Email, Начислено бонусов, Прошло сценарий (в штуках и %), Выручка.
- **График:** Визуализация соотношения Выручки и количества Покупок по дням. Ниже сохраняется блок с текстовым описанием логики и условиями отбора.



Таб «Схема»

Таб предоставляет drag-and-drop интерфейс для построения маршрута клиента.

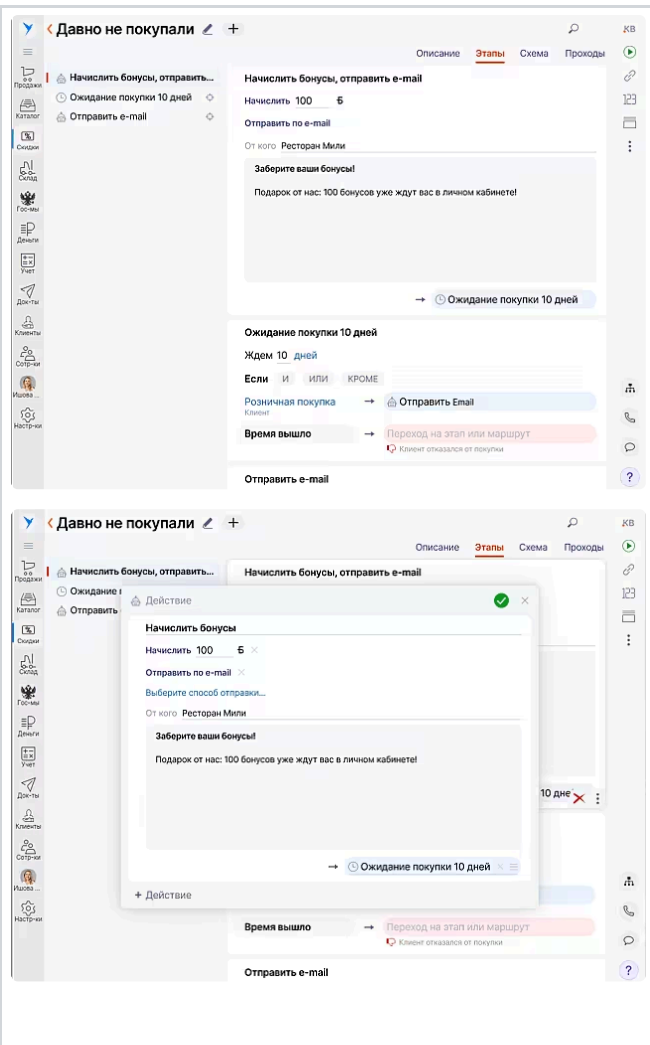
- **Пустое состояние:** При первом входе выводится подсказка, объясняющая назначение базовых блоков: Действие, Условие, Ожидание.
- **Добавление узлов:** По клику на кнопку «+» открывается меню. Можно выбрать быстрое действие (Начислить бонусы, Назначить скидку, Отправить) или добавить базовый блок (Действие, Условие, Ожидание).
- **Настройка блока «Действие»:** В модальном окне узла можно задать параметры действия (например, количество бонусов). Кнопка «+ Действие» позволяет сгруппировать несколько независимых действий (например, начислить баллы и отправить письмо) в рамках одного этапа.
- **Настройка блока «Ожидание»:** Блок позволяет настроить паузу (например, «Ждем 10 дней»). Имеет встроенную логику ветвления:
 - При наступлении выбранного события (например, «Розничная покупка») клиент направляется по одной ветке маршрута.
 - При срабатывании условия «Время вышло» — по другой (например, ветка отказа/напоминания). У каждого исхода настраивается свой переход на следующий этап.



Таб «Этапы»

Таб предоставляет альтернативный визуальному редактору вид настройки маршрута. Интерфейс разделен на две части:

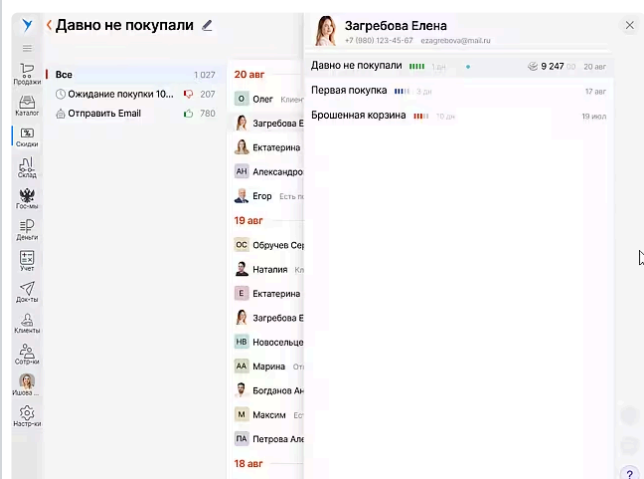
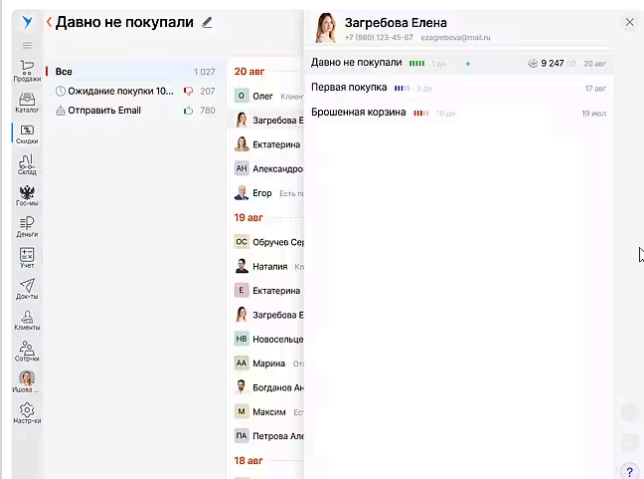
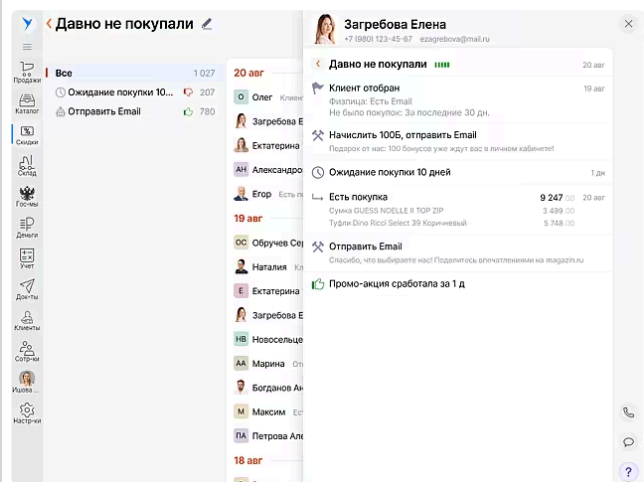
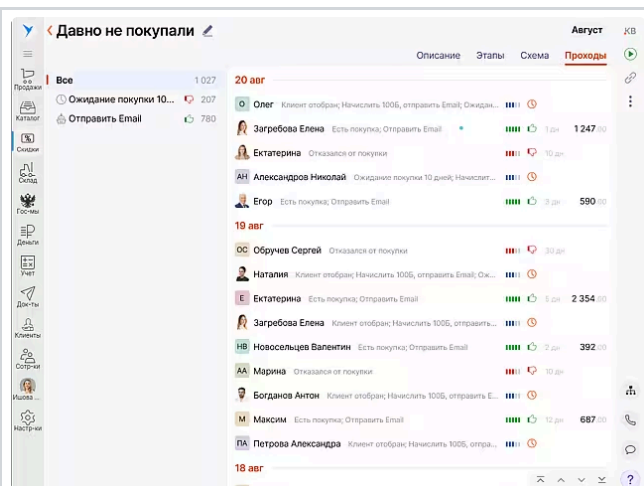
- **Левая панель:** Содержит линейный список всех созданных в сценарии блоков.
- **Правая панель:** Отображает детальную конфигурацию выбранного в левом списке блока.
- **Редактирование:** При клике на конкретное действие или условие открывается окно настроек, полностью идентичное тому, что используется во вкладке «Схема». Это позволяет быстро вносить изменения в параметры действий (сумма бонусов, текст письма и т.д.) без необходимости работать с визуальным графом.



Таб «Проходы»

Данный раздел предназначен для детального анализа истории срабатывания сценария по каждому клиенту.

- **Сводка по блокам (Левая панель):** Отображает агрегированную статистику успехов и отказов (положительные и отрицательные исходы) в разрезе ключевых блоков маршрута (например, сколько человек отвалилось на этапе «Ожидание», а сколько успешно получили «Email»).
- **Лента событий (Правая панель):** Хронологический список клиентов, сгруппированный по датам (например, «20 авг», «19 авг»).
- В каждой строке выводится: аватар, ФИО клиента, краткая текстовая выжимка пройденного пути, иконка финального статуса (успех/отказ), общее время нахождения в сценарии и сумма выручки (если была совершена покупка).
- **Детализация прохода (Карточка клиента):** При клике на строку клиента открывается боковая панель с детальной информацией:
 - В шапке отображаются контактные данные клиента (ФИО, телефон, email).
 - Ниже строится вертикальный таймлайн (история) конкретного прохода: момент отбора, выполненные начисления, этапы ожидания и зафиксированные события (например, «Есть покупка» с указанием купленных товаров и суммы).
 - В карточке отображаются как уже завершенные (успешные или нет) проходы, так и те, где клиент в данный момент «завис» на этапе ожидания.
- **Глобальная история клиента:** По клику на стрелку «Назад» у сценария: открывается сводный реестр всех сценариев, в которых когда-либо участвовал данный клиент (с указанием статуса и выручки по каждому).



5. Условия лицензирования

В рамках текущей тарификации Лояльности - возможно в полном тарифе

6. Техническое решение

6.0 Основные работы

6.0.1.1 Триггерные события

Для достижения целей проекта разработчики прикладных областей должны предоставить описание своих объектов, их событий, и действий, а так же внедрить в прикладные сценарии отправку описанных событий.

▼ О системе триггеров

Для оптимальной работы проходчика маршрутов необходим механизм доставки различных фактов/действий клиентов в прикладных сценариях, на основе которых проходчик реализует логику перемещения по этапам маршрута каждого отдельного клиента.

Триггерные события и действия - это то, что позволит формализовать взаимодействие прикладного кода с механизмом маршрутов лояльности, благодаря чему мы получим возможность настраивать гибкие сценарии без необходимости каждый раз запрашивать доработки прикладных сценариев.

Контракт события/действия - json описание в отдельном [репозитории](#).

Каждый json служит инструкцией одновременно о том, какие данные необходимо передать из прикладной области для обработки события, и о том, каким образом эти данные могут быть обработаны.

В документации описаны допустимые поля и пример:

https://link.sbis.ru/customization/knowledge?published=true&mode=readList&article=bc5dc76a-aa42-4d97-88b5-532733b24ca8#toc_41b8af6a-f269-48c0-be26-5b8f1ab9a0e9

После того как описаны события, они должны быть закреплены за тем или иным объектом (каталогом).

https://link.sbis.ru/customization/knowledge?published=true&mode=readList&article=bc5dc76a-aa42-4d97-88b5-532733b24ca8#toc_adfac739-30f8-4f5c-a5a2-cd97108b7161

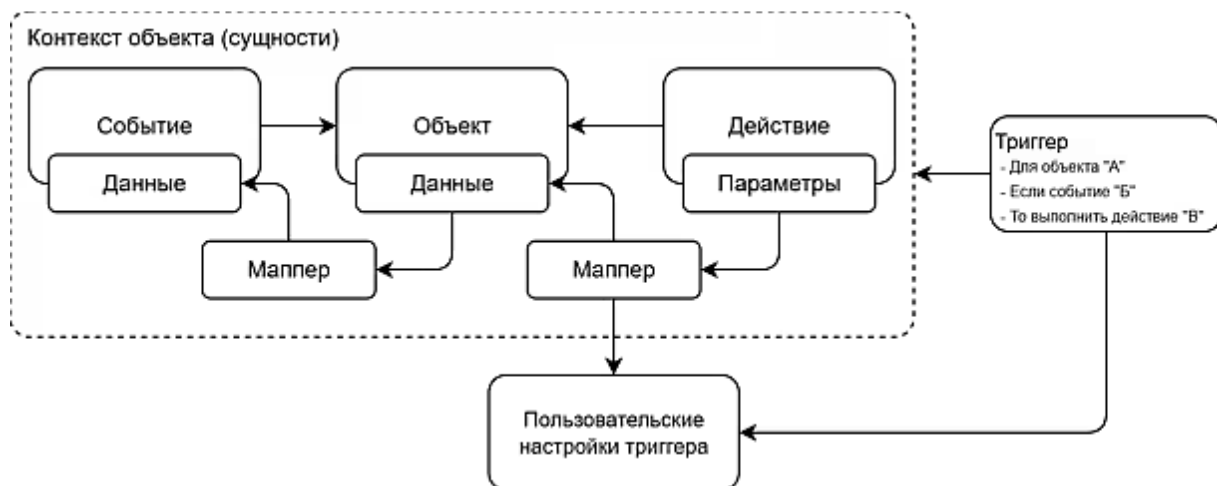


Схема взаимосвязи примитивов механики триггеров, и как мы используем их в проекте

Триггеры не будут настраиваться пользователем напрямую. Они будут создаваться при настройке каждого нового маршрута, и при срабатывании просто уведомлять проходчик о новых изменениях, на основе которых тот сможет принимать решение о перемещении клиентов по этапам маршрута. Т.е. **роль триггеров сугубо техническая.**

Очередной этап маршрута может оказаться **действием**, которое проходчик должен выполнить для каждого клиента, находящегося на этом этапе. Т.к. фактические изменения выполняются в прикладных областях, их вызов, так же как отправка событий, должен осуществляться в рамках строгого контракта. Описание триггерных действий позволит обеспечить этот контракт. Для запуска действий так же будут создаваться тех.триггеры без специального события для активации. Проходчик будет принудительно "поджигать" такие триггеры при необходимости выполнить связанное действие.

Настройка событий

Чтобы заработал список событий для запуска сценариев необходимо:

1. Добавить описание событий в виде JSON в [репозиторий триггеров](#).
 1. Внедрить отправку событий в прикладных сценариях.
2. Добавить недостающие объекты [в репозиторий](#), либо выбрать из существующих в [реестре триггеров](#)

События, поддерживаемые в рамках текущего проекта:

Объект/Событие(я)/Подробно:

Первый приоритет:

- [Посетитель сайта](#) saby get (частично)

События этого типа будем получать от механизма трекинга отдела Маркетинга.

▼ Подробное техрешение

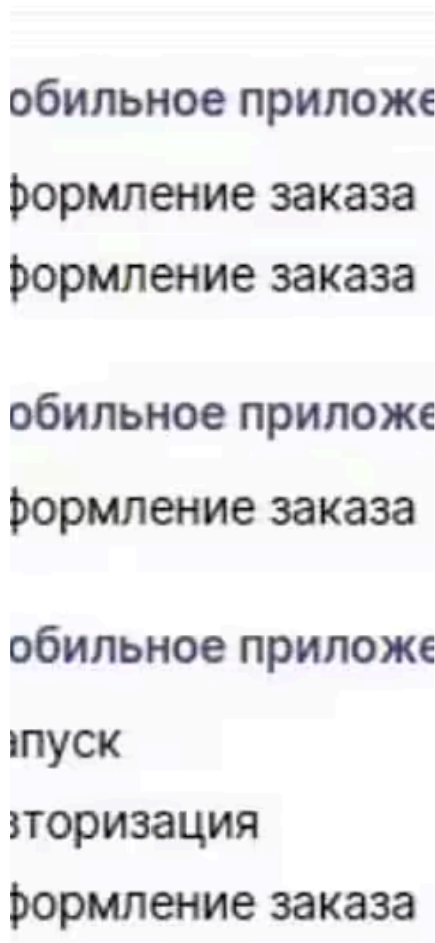
Для хранения и последующей постобработки информации о пользовательских действиях на сайте и в БМП расширяем механику достижения пользовательских целей на сайте в подсистеме трекинга.

<https://yandex.ru/support/metrika/ru/general/goals>

Сейчас факты открытия БМП, авторизации в нем, а так же оформления заказа из него реализованы по такой схеме — введены новые типы событий по посетителю в рамках сайта-БМП.

<https://project.saby.ru/uuid/8b350187-6258-4095-b08b-77f6fd0cbf9b/page/project-main>

Они же отображаются в карточке истории пути посетителя.



В рамках проекта вводим дополнительные новые события-цели для регистрации сервисом трекинга:

5 — открытие корзины

6 — размещение товара в корзине

7 — просмотр карточки товара

Прикладной стороной — функционалом работы с корзиной в БМП или с виджетом на сайте

при срабатывании этих пользовательских действий, осуществляется:

1. В случае виджета на сайте — вызов метод `js-api` скрипта трекинга `cnsmrSaby.goalReached()` - который мы сделаем публичным в рамках проекта. Он сам добавит в отправляемую пачку данных информацию о достигнутой цели в атрибут `Events`
2. В случае БМП — пополняем информацией о достигнутой цели массив, отправляемый в метод `ConsumerTracking.DeviceStat`

При разборе очередной пачки сервис трекинга, в случае, если были достигнуты

пользовательские цели, сохраняет `set` их идентификаторов в атрибутах визита, а так же все достигнутые указанные цели при отправке пачки статистики трекинга сохраняет в сыром виде в таблицу `events` на сервисе `consumer-stats`.

См. описание таблиц

<https://wi.sbis.ru/page/autodoc-database/services/AutodocTableInfo/8ccb59cf-34fb-4088-ba36-2a907c3001e5%2Fe46ea7ff-c9f1-47ec-b6a3-dfa9a0e1dd3d%2FVisit?v=https://online.saby.ru/page/knowledge-bases/94a1239e-a576-4871-98f7-3bf53c65f269?article=0b3ccdea-90bb-4715-a344-7ae733405bc0&published=null&mode=readL>

Данные из таблицы `events` используются для рендера в пути посетителя в методе `ConsumerTracking.ActivityList/4`

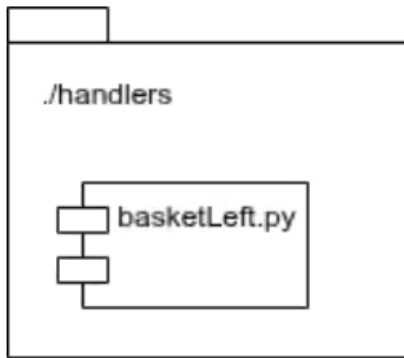
Алексеевич ID 297722589 ● Сейчас на сайте		
.sabyd.ru	13:11	Заинтересован
ящие документы		Визиты 128
.sabyd.ru		Клики 2 738
ботать ответ покупателя	:36	На сайте 41:56:50
ючить многосторонний договор	:29	Впервые 3 июня
Docs	:36	С поиска Google
тановить удаленный документ	4:31	
лировать документ в Saby Docs	:30	
фровать вложение	:07	
и документ	:30	
ерить статистику документов в Saby Docs	:30	
ласить к ЭДО из Saby Docs	:52	
тановить удаленный документ	:24	
лировать документ в Saby Docs	:22	
фровать вложение	:02	
ерить статистику документов в Saby Docs	:03	
ласить к ЭДО из Saby Docs	:06	
Docs	6:05	

Сбор информации о триггерных событиях

На основании собранной истории взаимодействия посетителей с сайтами и бмп можно построить ряд поведенческих паттернов и на основании их осуществить выброс триггерных событий.

Например — понять что было начато заполнение корзины но не достигнут целевой итог в виде заказа.

Предлагается на сервисе трекинга реализовать механику, позволяющую прочесть свежие данные и выполнить пользовательские обработки по собранной информации



Реализуем в виде метода координатора, который:

1. Начитывает данные с сервиса трекинга по визитам за последний час.

Читаем все визиты с указанными срабатываниями пользовательских целей

По ним начитываем информацию обо всех event-ах в рамках этого визита

2. По собранному массиву данных вызывает все пользовательские обработчики, лежащие локально в модуле с выполняемым методом в подмодуле ./handlers

Запускаем метод по планировщику каждые полчаса.

Обработчик — брошенная корзина

В качестве первого такого обработчика и образца для остальных — реализуем детекцию брошенной корзины.

Алгоритм определения:

1. В рамках визита есть достижение целей открытие корзины и размещения товара (5 и 6), но нет достижения цели — создание заказа (Lead = 1).
2. Визит завершен — его длительность от начала более 30 минут, а с момента последней активности (StartTime + RealDuration) прошло более 15 минут

По всем таким визитам определяем посетителя и вызываем срабатывание триггера по событию.

Обработчик — товар просмотрен

Алгоритм определения:

- В рамках визита есть достижение цели 7 - просмотр карточки товара.

Обработчик — товар добавлен в корзину

Алгоритм определения:

- В рамках визита есть достижение цели 6 - размещение товара в корзине.

События:

- ▼ [Первый заход](#), [Каждый заход](#) (на сайт) - готовы

Клиент впервые/в очередной раз заходит на сайт.

В контексте события определяется сайт, которых на выбор очень и очень много, что сильно портит UX. Вероятно будем ограничивать доступность фильтра по этому полю в интерфейсе маршрута.

Набор данных для полезной нагрузки события изложен [в объекте](#)

Гайдабура Олег - Мария Коретникова тестер

- ▼ Товар просмотрен (new)

Возникает при открытии покупателем карточки товара.

Событие поднимается трекингом, на основе соответствующего действия клиента, полученного от прикладной области.

Набор данных для полезной нагрузки события расширяет изложенный [в объекте](#):

- ИД продажи (@Sale)
- ИД товара (@Nomenclature)
- Стоимость товара

- ▼ Добавление в корзину (new)

Возникает при добавлении товара в пустую корзину.

Событие поднимается трекингом, на основе соответствующего действия клиента, полученного от прикладной области.

Набор данных для полезной нагрузки события расширяет изложенный [в объекте](#):

- ИД продажи (@Sale)
- ИД товара (@Nomenclature)
- Стоимость товара

▼ Брошенная Корзина (new)

Возникает, если клиент добавил товары в корзину, но не осуществил заказ за отведенное время.

Событие поднимается трекингом, при нахождении в действиях клиента паттерна: Товар добавлен в корзину более 30 минут назад, заказ НЕ был оформлен и корзина НЕ была очищена.

При появлении такого паттерна активности для клиента по этому клиенту будет поднято триггерное событие.

Набор данных для полезной нагрузки события:

- ИД Клиента. (@Лицо в локальной схеме)
- Состав корзины (желательно с @Номенклатура локальной схемы)
- Сумма корзины

• Продажи розницы

▼ Совершена покупка (new)

Новое событие, реализуемое силами команды Розницы.

Возникает при успешном закрытии розничной продажи.

Ключевые моменты по пунктам:

- Факт закрытия любой продажи: розница, престо, салон, отель, десктопы (втч МП)...
- События по продажам **в десктопах и МП** поднимаются в момент импорта в онлайн.
- В остальных случаях событие должно отправляться в момент окончательного успешного закрытия продажи (успешной оплаты).
- Для продаж с **чеком намерения** - **в момент подтверждения оплаты?**
- Для "Покупок" (выкупаем что-то у клиента) событие **не поднимается**.
- Набор данных для полезной нагрузки события:
 - ИД продажи (@Sale)
 - ИД покупателя (@Лицо), если доступно
 - Сумма продажи
 - ИД точки продаж (Наверное @Лицо)
 - Приложение-источник; розница, престо, салон, отель, киоск?. Онлайн-оффлайн не разграничиваем.
 - "SaleName"? - (вроде-бы розница формирует автоматическое текстовое описание продажи из состава товаров)

Второй приоритет:

• [Клиент](#)

▼ [Создание](#)

Требуемое триггерное событие уже реализовано, но вызывается при создании любого контрагента (втч юр.лиц итд).

Через интерфейс начального этапа маршрута мы сможем неявно допараметризовать его применение, чтобы ограничить срабатывание именно покупателями розницы (Клиент.Тип == 2?)

Данные о клиенте (в частности Тип) доступны в триггере [через Маппинг](#).

Третий приоритет:

• [Карта](#) (new)

▼ [Выдана](#) (new)

Отправляется при выдаче новой дисконтной карты клиенту.

Набор данных для полезной нагрузки события:

- Клиент (@Лицо)
- UUID Персоны

- ИД Вида карты (@ВидКарты)
- ИД карты (@Карта)
- Номер карты

Внедряемся в БЛ выдачи карты, отправляем уведомление.

- Бонусы (new)

- ▼ Начисление (new)

Отправляется при начислении бонусов клиенту, **кроме ручных пополнений**

Набор данных для полезной нагрузки события:

- Клиент (@Лицо)
- UUID Персоны
- Сумма начисления
- ИД Вида карты (@ВидКарты)
- ИД карты (@Карта)
- ИД продажи-источника
- ИД бонусной программы (@ВидЦены)
- Флаг реферальных бонусов

Внедряемся в Bonus.ChangeHandler. Отправляем уведомление в случае начисления бонусов не вручную.

Операции через оффлайны завязаны сюда же

- ▼ Списание (new)

Отправляется при списании бонусов, **кроме ручных списаний**

Набор данных для полезной нагрузки события:

- Клиент (@Лицо)
- UUID Персоны
- Сумма списания
- ИД Вида карты (@ВидКарты)
- ИД карты (@Карта)
- ИД продажи-источника
- ИД бонусной программы (@ВидЦены)

Внедряемся в Bonus.ChangeHandler. Отправляем уведомление в случае списания бонусов не вручную.

Операции через оффлайны завязаны сюда же

- ▼ Сгорание (new)

Событие нужно для настройки действий "за произвольное N дней до сгорания бонусов".

В таблице Карта введем новую колонку для даты ближайшего сгорания бонусов по карте. При каждом перерасчете баланса (при совершении операций с бонусами) будем обновлять в этой колонке дату такого сгорания.

Разрабатываем БЛ метод, который будет возвращать список клиентов и карт с датой и суммой сгорания бонусов.

Через шедулер на онлайн заводим ежедневную задачу, которая будет запрашивать список клиентов через описанный выше БЛ, и поднимать по каждому клиенту триггерное событие о сгорании бонусов через (ДатаБлижайшегоСгорания - Now) дней.

Набор данных для полезной нагрузки события:

- Клиент (@Лицо)
- UUID Персоны
- Сумма сгорающих бонусов
- ИД Вида карты (@ВидКарты)
- ИД карты (@Карта)
- ИД продажи-источника (по которому было сделано начисление)
- ИД бонусной программы (@ВидЦены)

- Дней до сгорания?
- Промокод (new)
 - ▼ Выдан (new)

Отправляется при выдаче промокода за активность

Набор данных для полезной нагрузки события:

 - Клиент (@Лицо)
 - UUID Персоны
 - Сумма сгорающих бонусов
 - ИД Вида карты (@ВидКарты)
 - ИД карты (@Карта)
 - ИД продажи-источника (по которой выдется промокод)
 - ИД типа промокода (@ВидКарты)
 - ИД промокода (@Карта)
 - ИД связанной акции (@ВидЦены)
 - Номер промокода

Внедряемся в `promocode.activity.get_by_activity`. Уведомляем при выдаче промокода за активность.

В оффлайнах работает как прокси-метод, досылка при импорте тоже проходит через `get_by_activity`.
 - ▼ Применен на продаже (new)

Отправляется при закрытии розничной продажи, по которой списывается промокод.

Набор данных для полезной нагрузки события:

 - Клиент (@Лицо)
 - UUID Персоны
 - Сумма сгорающих бонусов
 - ИД Вида карты (@ВидКарты)
 - ИД карты (@Карта)
 - ИД продажи-источника (в которой списан промокод)
 - ИД типа промокода (@ВидКарты)
 - ИД промокода (@Карта)
 - ИД связанной акции (@ВидЦены)
 - Номер промокода

Внедряемся в `RetailSaleDoc.Activate->_process_apply_promocode`. Отправляем уведомление при обработке применения промокода.

Для оффлайн продаж искать запись промокода при импорте в онлайн

Настройка действий

Чтобы добавить новое действие, необходимо сформировать файл в формате JSON, который описывает действие. В [репозитории](#) все действия находятся в директории **base/actions**. Помимо базовых действий прикладная область может обладать своими уникальными действиями, которые допустимы к тому объекту, который описывается.

https://link.sbis.ru/customization/knowledge?published=true&mode=readList&article=bc5dc76a-aa42-4d97-88b5-532733b24ca8#toc_a01472aa-933c-46d8-9f7a-7b954079feef

Чтобы заработал список действий необходимо:

1. Добавить описание действий в виде JSON в [репозиторий триггеров](#).

1. Реализовать БЛ метод действия в прикладной области.

Этот БЛ метод будет принимать в качестве единственного аргумента Record, содержащий полезную нагрузку, описанную в объекте.

Для возможности задавать в интерфейсе настройки действия значения, которых нет в контексте, от команды триггеров планируем доработку, которая позволит описать требуемые поля ввода для действий, вплоть до конкретных компонентов-лукапов.

2. Зафиксировать доступность действия на объектах, которые используются в конфигурации.

Действия, поддерживаемые в рамках текущего проекта:

Первый приоритет:

▼ Отправить сообщение (почта, смс, уведомлении мессенджеры и пр.)

Действия уже существуют. Для работы достаточно добавить их в обработчики событий для описываемых в рамках проекта действий.

1. <https://git.sbis.ru/customization/triggers-data/-/blob/development/base/actions/SocNetAggregation.Send.json>
2. <https://git.sbis.ru/customization/triggers-data/-/blob/development/base/actions/Notification.Send.json>

▼ Изменить баланс бонусов (new)

По ИД клиента выполняем списание/начисление в зависимости от знака суммы начисления (или отдельного флага)

Параметры действия в UI:

- Сумма списания/начисления
- Комментарий

Выполнение:

По набору переданных параметров конфигурируем вызов Bonus.ChangeHandler, выполняем списание/начисление.

Второй приоритет:

▼ Добавить/исключить в папку (new)

Заводим новые БЛ метод для обработки триггерных действий.

Параметры действия в UI:

- Папка клиентов (в какую включить/из какой исключить)
+ метод для лукапа папок

Выполнение:

Непосредственно обработка выполняется методами БЛ Гаврилова М.

Третий приоритет:

▼ Выдать промокод (new)

Выдать можно только индивидуальный промокод.

Оформляем выдачу по ИД клиента и ИД типа промокода.

Параметры действия в UI:

- Тип промокода (только индивидуальные)
- Настройки генерации

Выполнение:

В нем конфигурируем вызов IndividualPromoCodeEmission.Generate. Рассылки промокодов не задействуем, до покупателя промокод доставляем механизмами маршрутов.

Если в маршруте не настроено никакое уведомление клиенту, ПК будет сгенерен "в пустоту" - на это пока согласны.

▼ Назначить скидку (new)

Назначает клиента на некую персональную акцию (Дополняет условие "Клиент" у акции)

Допускаются Скидки, Подарки, Бонусные программы, для которых либо не еще не задано условие "Клиентам", либо указаны конкретные лица.

Параметры действия в UI:

- Акция (которая будет персонльно назначена клиенту)
+ метод для лукапа акций (с фильтрацией применимых акций)

Выполнение:

Конфигурируем корректный вызов Promotion.Update для назначения акции на клиента.

Если выбрали акцию без условия в маршруте, а затем назначили владельцев карт - действие должно падать с ошибкой. Ошибка будет отображаться в окне прохода по клиенту.

Возможно стоит схитрить, и сделать это действие почти идентичным "переместить в папку", а для настройки будем предлагать только те скидки, для которых в условии "Клиентам" указаны папки(единственная папка).

▼ Выполнить спк код

Действие уже существует. Для его работы достаточно добавить его в обработчики событий для описываемых в рамках проекта действий.

Обработка ошибок и отладка

В случае возникновения ошибок при выполнении БЛ-обработчиков триггерных действий, проходчику маршрутов будет передан факт неудачного выполнения действия. Ошибки этапа-действия будут отображаться в карточке прохода.

Поиск и исправление системных ошибок предлагается выполнять классическими средствами – через облачные логи и профилирование, т.к. выполнение триггерного действия сводится к простому вызову метод БЛ.

6.0.1.2 Реестр (и карточка) сценариев лояльности

Страница с реестром

Страница предназначена для просмотра списка сценариев, управления с помощью быстрых действий без проваливания внутрь сценария и просмотра общей статистики проходов за выбранный период по каждому сценарию. Страница будет реализована в виде двухколоночного реестра (мастер-деталь). Слева (мастер) располагается иерархия сценариев, справа (деталь) – созданные сценарии в разрезе выбранного пункта иерархии.

Сценарии +		Найти	Август	KB
Все	Начисление бонусов	5	Давно не покупали	1 027 935
Каталог	Уведомления	4	Письмо с напоминанием	735 324
Скидки			Брошенная корзина	488 894
Склад			День рождения	1 543 1 193
Гос-чи			Бонусы при регистрации	3 651
Денег			Уведомление о начислении	570
Учет			Первая покупка	2 936 2 907
Док-ты			Сезонное зимняя обувь	1 701 1 621
Клиенты				
Сотр-ки				
Итого ...				
Настройки				

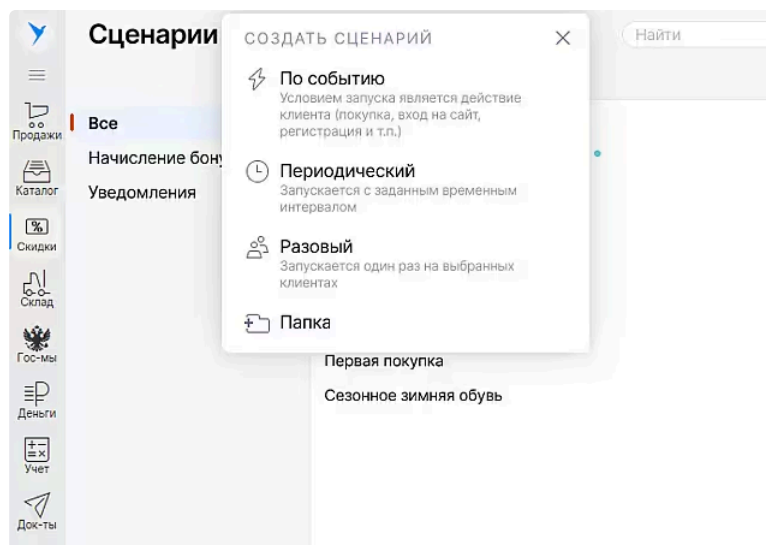
Доступны следующие операции над записью:

- удаление (для сценариев дополнительно будет поддержано массовое удаление);
- перемещение между папками (для сценариев дополнительно будет поддержано массовое перемещение);
- изменить название (актуально только для папок);
- показать историю (актуально только для сценариев).

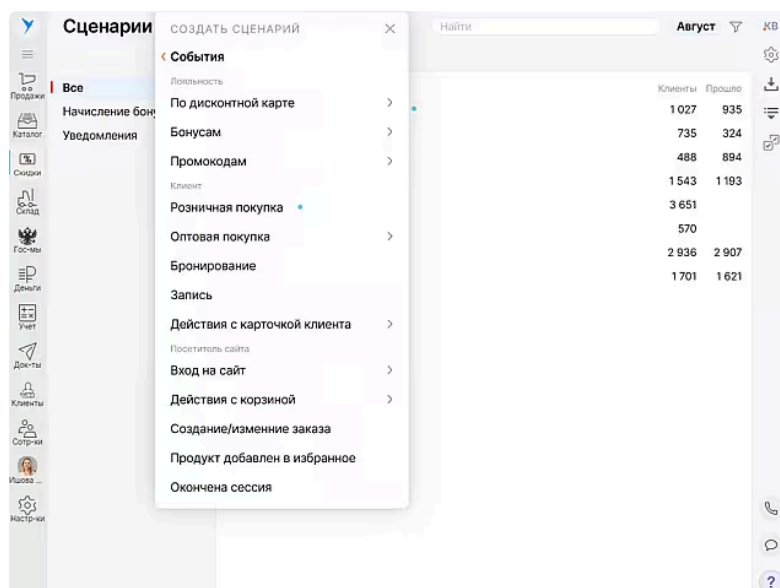
Страница будет реализована с использованием компонентов платформы маршрутов согласно [документации](#). В качестве мастера будет использоваться [Route/Registry/master:View](#), в качестве детали – [Route/Registry/list:View](#). Подробную декомпозицию см [раздел](#). Для загрузки данных для построения страницы будет использоваться прикладная обертка над списочным методом платформы маршрутов [RouteService.List](#). В прикладной обертке задаем права доступа к методам, зона – **PF-LoyaltyProgram-PUB**.

Кнопка создания сценария

В рамках проекта платформа маршрутов предоставит новый компонент для создания сценария с возможностью выбора типа – По событию, Периодический и Разовый.



Для типа По событию будет предусмотрен дополнительно выбор события.



Для создания сценария/папки будет реализована прикладная обертка над методом платформы маршрутов [RouteService.Create](#).

Удаление

Для перемещения сценария/папки будет реализована прикладная обертка над методом платформы маршрутов [RouteService.Delete](#).

Перемещение

Для перемещения сценария/папки в другую папку будет реализована прикладная обертка над методом платформы маршрутов [RouteService.ChangeFolder](#).

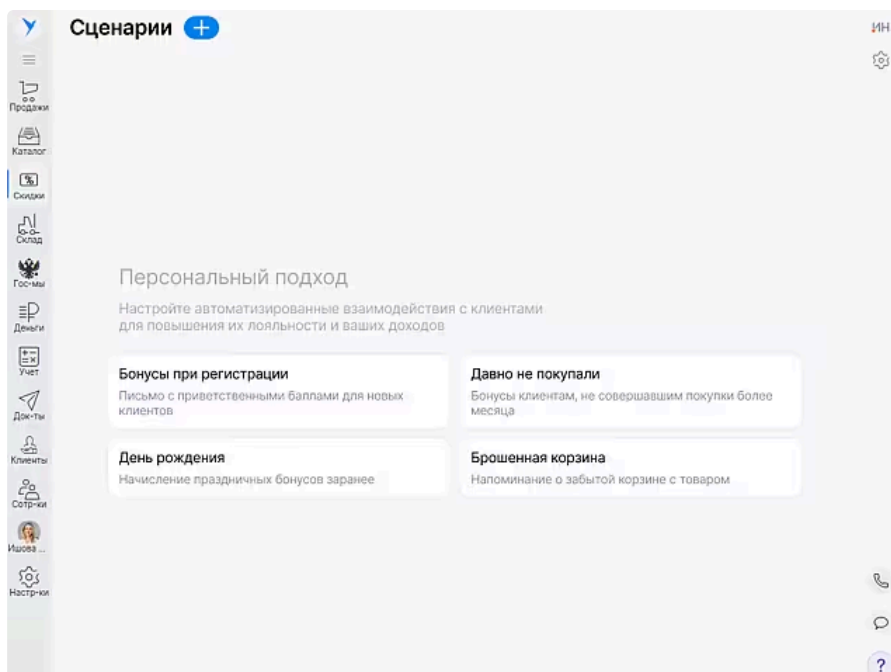
История

Формирование истории изменений используем встроенный механизм платформы маршрутов согласно документации [RouteService.Update](#).

Специфичных для маршрутов лояльности сценариев ведения истории на момент составления ТЗ не предполагается, однако при необходимости мы сможем модифицировать базовое поведение платформы маршрутов с помощью [соответствующего публичного API](#).

Первый вход

Для отображения состояния первого входа будет использоваться платформенный механизм для страниц - [emptyViewConfig](#). Для определения состояния Первый вход реализуем метод БЛ (по аналогии с существующими методами для других страниц лояльности, например, `Promotion.HasAvailable`). Метод будет проверять, есть ли уже созданные или деактивированные сценарии на аккаунте.



Страница сценария

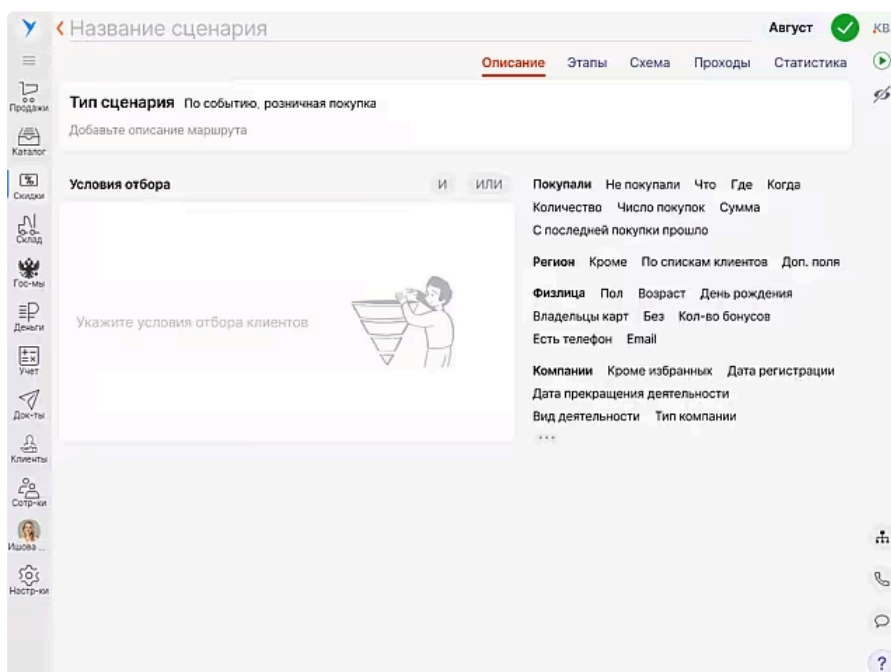
Страница маршрута разделена на вкладки - Описание, Этапы, Схема, Проходы, Статистика. Редактирование страницы будет реализовано через платформенный механизм [formMethodConfig](#). Для чтения и сохранения записи настроек сценария будут реализованы прикладные методы на основе методов платформы маршрутов [RouteService.Read](#) и [RouteService.Update](#).

Вкладка Описание

Вкладка Описание будет релизована на основе компонентов платформы маршрутов, согласно [документации](#), декомпозиция см [раздел](#). Будет предусмотрен следующий набор блоков:

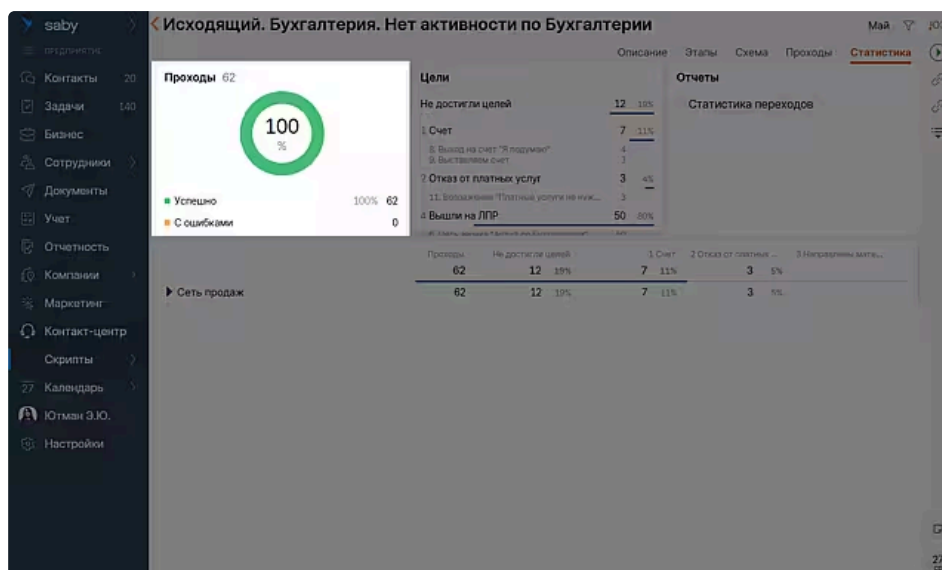
- Описание (блок платформы маршрутов)
- Тип сценария (блок платформы маршрутов, реализуется в рамках [проекта](#))
- Ответственный (блок платформы маршрутов)
- Условия отбора (блок платформы маршрутов, реализуется в рамках [проекта](#))

При добавления маршрута по событию "создание клиента", обязательно должен быть выставлен фильтр для тех.триггера по типу клиента, т.к. исходно событие отправляется при создании вообще любых контрагентов



В корне результата выполнения [метода](#) чтения сценария лежат все данные для отображения элементов текущей вкладки, отдельные методы БЛ не требуются. Привязку полей в блокам, см [документацию](#). Блоки с типом сценария и условиям отбора будут реализованы по параллельному [проекту](#).

В режиме чтения на странице будет отображаться виджет со статистикой по прохождениям, см скрин. Для получения данных для виджета будет использоваться прикладной метод, ор=бертка над методом платформы маршрутов [RouteStat.GetGoalStepStat](#).

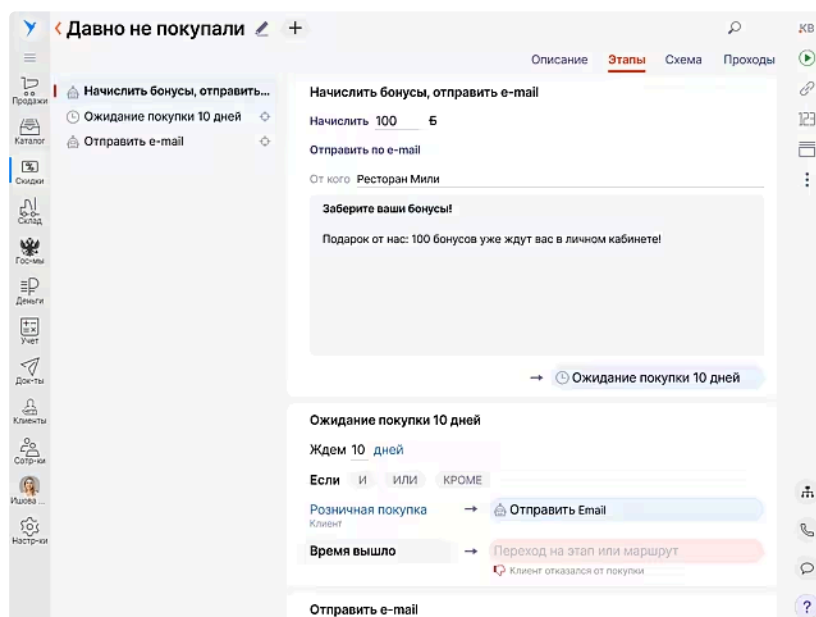


Вкладка Этапы

Вкладки Этапы и Схема предназначены для просмотра и редактирования маршрутной части сценария. Обе вкладки работают с одной маршрутной моделью и отображают:

- этапы маршрута;
- переходы между этапами.

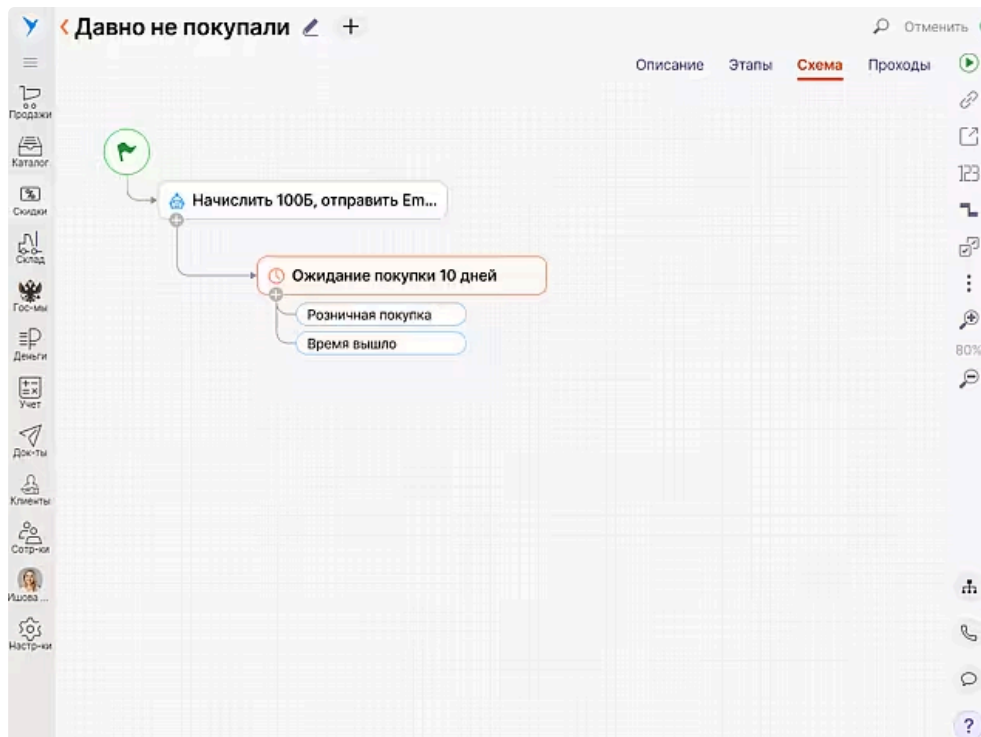
Вкладка Этапы будет релизована на основе компонентов платформы маршрутов, согласно [документации](#), декомпозиция см [раздел](#).



В корне результата выполнения [метода](#) чтения сценария лежат все данные для отображения элементов текущей вкладки, отдельные методы БЛ не требуются.

Вкладка Схема

Вкладка Схема будет релизована на основе компонентов платформы маршрутов, согласно [документации](#), декомпозиция см [раздел](#).



В корне результата выполнения [метода](#) чтения сценария лежат все данные для отображения элементов текущей вкладки, отдельные методы БЛ не требуются.

Вкладка Проходы

Вкладка Проходы будет релизована на основе компонентов платформы маршрутов, согласно [документации](#), декомпозиция см [раздел](#). Вкладка предназначена для просмотра проходов по сценарию с возможностью фильтрации. Дополнительно в мастере можно просмотреть статистическую информацию. Для получения статистики по этапам (мастер) будет реализована прикладная обертка над методом платформы маршрутов [RouteStat.GetStat](#), для получения списка клиентов - [RouteService.TraceList](#).

Дата	Клиент	Описание	Статус	Сумма
20 авг	Олег	Клиент отображ; Начислить 100Б, отправить Email; Ожидан...	Прошел	
	Загребова Елена	Есть покупка; Отправить Email	Прошел	1 247.00
	Екатерина	Отказался от покупки	Не прошел	
	АН Александров Николай	Ожидание покупки 10 дней; Начислит...	Прошел	
	Егор	Есть покупка; Отправить Email	Прошел	590.00
19 авг	ОС Обручев Сергей	Отказался от покупки	Не прошел	
	Наталия	Клиент отображ; Начислить 100Б, отправить Email; Ож...	Прошел	
	Екатерина	Есть покупка; Отправить Email	Прошел	2 354.00
	Загребова Елена	Клиент отображ; Начислить 100Б, отправит...	Прошел	
	НВ Новосельцев Валентин	Есть покупка; Отправить Email	Прошел	392.00
	АА Марина	Отказался от покупки	Не прошел	
	Богданов Антон	Клиент отображ; Начислить 100Б, отправить E...	Прошел	
	М Максим	Есть покупка; Отправить Email	Прошел	687.00
	ПА Петрова Александра	Клиент отображ; Начислить 100Б, отпра...	Прошел	
18 авг				

Добавление прикладного маршрута

Для сценариев лояльности в Route регистрируем новые значения:

Поле	Значение
Category	Loyalty

Registry	manual - ручной запуск event - по событию planed - по расписанию
----------	--

В Trace регистрируем новые значение:

Поле	Значение
pass_type	6 - маршруты лояльности

6.0.1.3 Схема сценария дополняется этапом "Действие"

<https://link.sbis.ru/customization/knowledge?published=true&mode=readList&folder=82f25661-c8cf-4bd5-8398-f9d7ecd6a968>

[Концепт этапа действия в регламентах](#)

Этап действие - это специальный тип узла, в котором настраиваются триггерные действия с правилами перехода на следующие этапы. На этапе может быть настроено одно и более действий. Редактирование действия происходит через компонент сервиса триггеров.

Действие - это функция, зарегистрированная в триггерах и входит в структуру прикладного объекта. В сервисе маршрутов хранится **UUID** действия и декларация параметров для запуска. Выполнение происходит в сервисе триггеров и запускается через метод [Trigger.Notify/4](#) в платформе маршрутов.

Весь список действий, которым конфигурируется этап описаны в [репозитории](#) в директории **base/actions**. Помимо базовых действий прикладная область может обладать своими уникальными действиями, которые допустимы к тому объекту, который описывается.

Модель узла

Поле	Тип	Описание
Action	Uuid	UUID действия. Передается в поле Data.TriggerID

Конфигурация этапа

По месту внедрения триггеры реализуют 2 основных механики:

1. Подключение компонента с набором действий

...//пример на wasaby, допускается и react вариант компонента

```

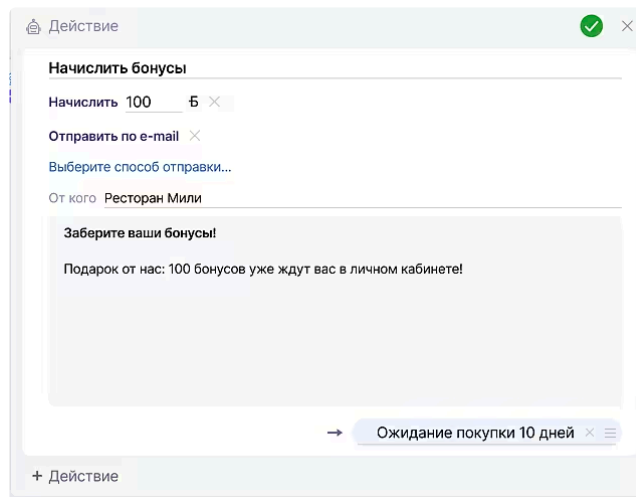
<Triggers.layout.Card
  options="{{options}}"
  context="{{context}}"
  saveCard()
  removeCard()
  onSaveCallback()
  onChangeCallback()
  onRemoveCallback()
/>
...

```

где

- options - сервисные параметры отображения компонента триггеров
 - options.payload - перечень полей для настройки условий и использования подстановок в действиях. Если объект документ, то payload рекомендуется оставить от документа и поджигать через Notify с передачей event
 - options.payload.NAME - внутри можно заявить required:true для обязательного заполнения условия, рисуется при открытии layout.Card. При заявлении собственных полей они должны иметь либо формат триггеров

```
{
```



```

"CustomType": "ctRESPONSIBLE",
"DisplayTitle": "Ответственный",
"IsParent": false,
"Localization": {},
"Name": "Сотрудник",
"Parent": null,
"Required": false,
"SymbolicName": "",
"Title": "Ответственный",
"Type": "atEMPLOYEE",
"Visible": true

```

либо формат дата-контекста:

```

// полный формат
{
  "Id": "CurrentAccount.Name",
  "Version": "26.3100",
  "Title": "Название",
  "Tooltip": null,
  "NodeType": "string",
  "Parent": "CurrentAccount",
  "Parent@": null,
  "Parent$": false,
  "Icon": null,
  "ChildNodeCount": 0,
  "ChildLeafCount": 0,
  "Responsible": null,
  "ResponsibleUUID": null,
  "Description": "Главная организация аккаунта",
  "Kaizen": null,
  "AdditionalInfo": {
    "MetaAttributes": {},
    "TypeInfo": {}
  },
  "AdditionalRecordInfo": {
    "Value": null
  }
}

```

- options.actions - помимо поставляемого набора действий в объекте, для отдельных действий можно задать правила
 - options.actions.exclude - перечень действий которые нужно исключить
 - options.actions.NAME - NAME(название действия), required(как пример), если нужно чтобы действие всегда отображалось, readonly(действие доступно только на чтение)
- context - дата-контекст для отрисовки полей в условиях + контекст объект и события
 - *context.object - объект(тип документа) куда подключается layout.Card, используем для endpoint'a карт данных + для выборки в БД
 - context.event - событие по которому будет выполняться перечень условий и действий. Механизм вызова триггеров Trigger.Notify. Если событие не передано, но нужно выполнить конфигурацию в момент времени, то вызывается Trigger.Notify с передачей идентификатора конфигурации из onSaveCallback()
 - context.phases, context.regulations - доп. информация для построения правильного перечня действий, полей условий и подстановок карт
 - context.id - поставляется прикладному разработчику при сохранении

- o context.callback - метод который триггеры должны вызвать по факту выполнения всех действий. В callback метод передаем гарантированно 2 параметра:

1. пейлоад триггера, с которым он поджегся

2. Статус обработки действий(текст ошибки, статус выполнения метода, статус действий)

Для таких триггеров фиксируем отдельный тип триггера - обособленные(произвольная конфигурация).

При встраивании по месту используется компонент layout.Card который при сохранении возвращает id конфигурации в onSaveCallback функции. При проигрывании данный id передается внутри context в поле context.id. При базовой конфигурации компонент представлен в месте интеграции как набор условий и перечень действий

При необходимости перечень условий может быть отключен и компонент поставляет исключительно набор действий. Набор действий поставляется конфигурации аналогично автошаблонам https://git.sbis.ru/customization/triggers-data/-/blob/development/objects/CustomDocTemplate.json?ref_type=heads которые не зависят от типа.

- "SocNetAggregation.Send"+"Notification.Send.Detached", - отправить
- "Object.Create", - создать
- "Object.AddComment", - добавление комментария
- "History.Write", - запись в историю
- "Code.Execute", - выполнить код
- "Http.Request", - http запрос
- "Document.Mark" - пометка
- "Calendar.Create" - событие в календарь
- пр. действия, добавляемые триггерами по документам в одностороннем порядке



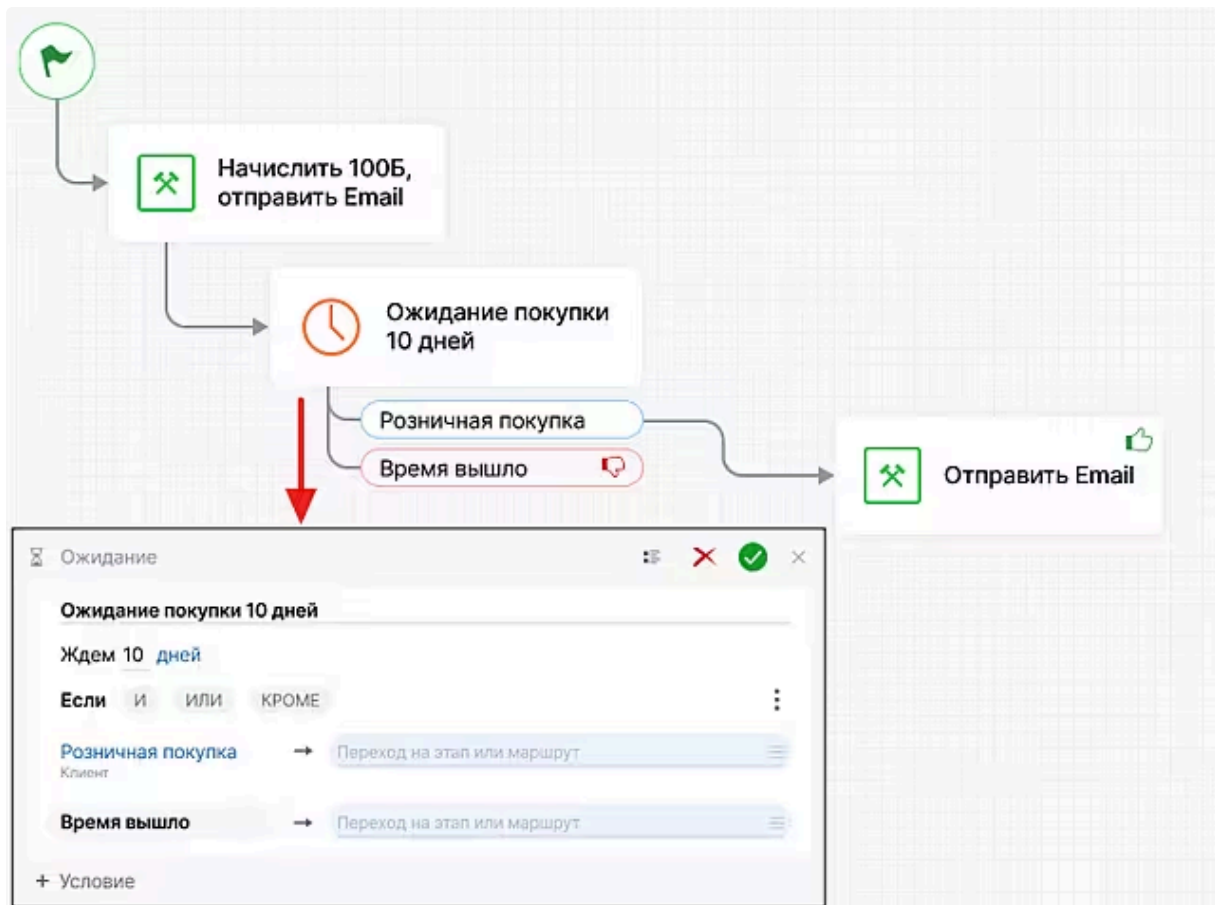
При передачи в _options.mount название действия, компонент загрузит действие, которое нельзя удалить. Такое действие является обязательным для заполнения. У каждого действия свой набор параметров для mount.



См. в [Т3 Исполнение этапа "Действие" в авто-проходчике](#)

6.0.1.4 Схема сценария дополняется этапом "Ожидание"

Позволяет организовать возможность ожидания прикладного события в течении указанного времени. С дальнейшим переходом на этап в случае если события дождались, и перехода на другой этап если не дождались.



Реализация выполняется в рамках проекта ["Таймер внутри маршрута"](#)

💡 См. в [ТЗ Исполнение этапа «Ожидание» в авто-проходчике](#)

6.0.2.1 Помещение клиента на маршрут

Сценарии запуска



▼ Схема запуска

```

1  ---
2  config:
3    theme: default
4  ---
5  sequenceDiagram
6
7    participant A as Пользователь/<br>прикладной сервис
8    participant B as Сервис Триггеров
9    participant C as Сервис расписаний
10
11   participant D as Сервис Маршрутов
12   participant E as Прочий сервис
13   participant H as Сервис выборки
14   participant F as DWC
15
16   Note over A,F: Запуск прохода маршрута
17   alt
18     Note left of B: Разовый старт
19     A ->>+ D: Запуск маршрута <br> Route.StartTrace(Route)

```

```

20      D --> H: Запуск задачи на отбор клиентов выборки<br>
CoreV3.StartSelection/4(ID выборки, Callback, Placeholder,
CallbackError)
21      H --> D: Статус о запуске
22      H --> F: Список клиентов для Запуск сценариев авто-проходчика
Route.StartTrace(Route, Clients, Placeholder)
23      D -->- A: Статус о запуске прохода <br> Route, Trace, Person
24
25      else
26          Note left of B: Старт по событию
27          E --> B: событие, на которое<br>настроен триггер или таймер
28          B -->+ D: Запуск маршрута <br> Route.StartTrace(Route, clients
or Null)
29      D --> H: Запуск задачи на отбор клиентов выборки<br>
CoreV3.StartSelection/4(ID выборки, Callback, Placeholder,
CallbackError)
30      H --> D: Статус о запуске сбора клиентов
31      D -->- B: Статус о запуске прохода <br> Route, Trace, Person
32      H --> F: Список клиентов для Запуск сценариев авто-проходчика
Route.StartTrace(Route, Clients, Placeholder)
33
34      else
35          Note left of B: Старт по таймеру
36          C --> C: Срабатывает условие таймера для запуска
37          C -->+ D: Запуск маршрута <br> Route.StartTrace(Route)
38      D --> H: Запуск задачи на отбор клиентов выборки<br>
CoreV3.StartSelection/4(ID выборки, Callback, Placeholder,
CallbackError)
39      H --> D: Статус о запуске сбора клиентов
40      D -->- C: Статус о запуске прохода <br> Route, Trace, Person
41
42      H --> F: Список клиентов для Запуск сценариев авто-проходчика
Route.StartTrace(Route, Clients, Placeholder)

```

Модуль **запуска прохода маршрута** отвечает за инициирование прохождения маршрутов (Trace) для клиентов по нескольким сценариям: разовый, по событию, по таймеру.

Общий набор операций

- Вызывается клиентский метод **Route.StartTrace/2** для подготовки запуска автопрохода. В методе вычитываются клиентские глобальные переменные для наложения фильтров. После чего обращается к сервису выборов, вызывая новый метод **CoreV3.StartSelection/4**:
 - ID - идентификатор выборки (сохраненной на этапе дизайн-тайма в маршрут)
 - Callback - вызываемого метода **RouteService.StartTrace**
 - Placeholder - параметрами для запуска callbacks
 - CallbackError - метод сообщения об ошибке **RouteService.StartTraceError**.
- Сервис выборов запускает задачу на отбор клиентов по указанной выборке и по завершении передает в сервис Маршрутов список клиентов по указанному callbacks, для которых необходимо стартовать маршрут
- Сервис выборов по пачкам клиентов запускает через DWC задачу для метода **RouteService.StartTrace(Route, Clients, Placeholder)**.

Разовый запуск прохода

1. Пользователь или внешний прикладной сервис иницирует разовый запуск маршрута, передавая идентификатор маршрута.
2. Запускается общий набор операций

3. Возвращается статус, что проходы запущены

Запуск по внешнему событию (триггер)

При настройке сценария на интерфейсе, формируется конфигурация триггера с условиями срабатывания по событию. При сохранении сценария в системе триггеров, для запуска настроенного сценария, создается **скрытый не запущенный триггер**, который существует пока сценарий не будет удален. При запуске сценария на исполнение, триггер запускается на исполнение.

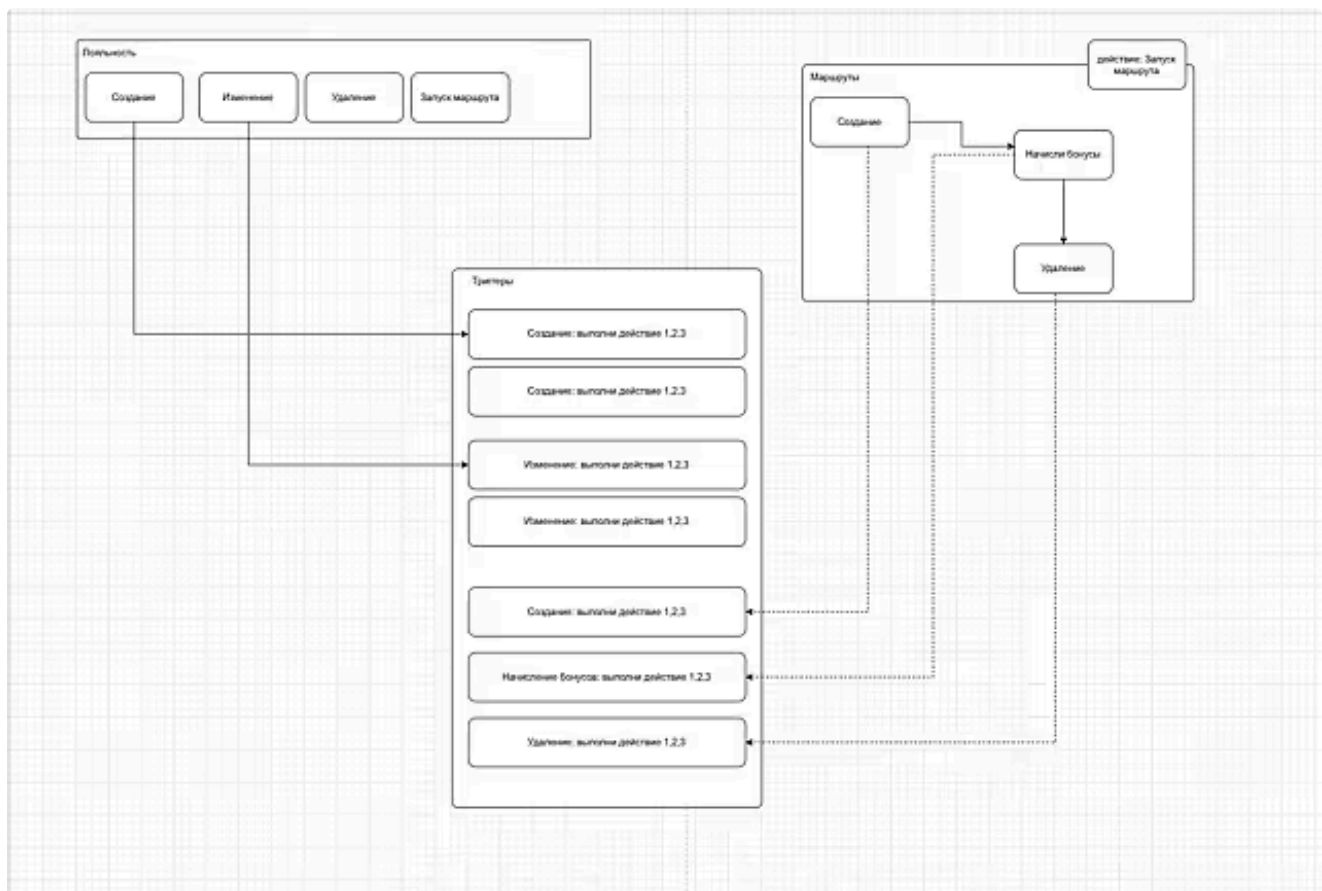
Если в интерфейсе сценария вносится изменение в условия срабатывания, происходит редактирование созданного триггера.

При срабатывании триггер запускает общий набор операций для запуска.

Если срабатывает триггер настроенный на конкретного клиента, он передает в метод **Route.StartTrace/2** конкретное лицо для запуска.

Если срабатывает триггер для всех клиентов, второй параметр остается Null и маршрут запускается для все выборки.

Прикладные сервисы не связаны с триггерами, создаваемыми сервисом маршрутов. Для управления маршрутом (запуск, остановка, статус) используются методы сервиса маршрутов по UUID маршрута.



Запуск по таймеру (плановые события)

<https://link.sbis.ru/article/7a20c7ca-316f-4ee2-9956-9efedca69c8a/242c4c51-d698-49a5-97a3-dc1260977cad>

Сервис плановых событий (Сервис расписаний) отслеживает настройки таймеров и контролирует наступление момента запуска, связанного с конкретным маршрутом и выборкой.

При наступлении заданного времени срабатывает условие таймера, и сервис расписаний фиксирует факт срабатывания этого условия для запуска маршрута.

Сервис плановых событий инициирует запуск маршрута, передавая в клиентский модуль Маршрутов сведения о маршруте.



Для каждой пачки клиентов (1000 клиентов) запускаются сценарии авто-проходчика, реализующие автоматическое прохождение маршрута на сервисе DWC.

Сервис выборки

Для работы с выборками вводим новые правила:

- Выборка не может быть запущена одновременно несколько раз. Для этого в выборке добавляется 3 новых параметра
 - Отложенный запуск - выборка сохраняется в сервисе и по идентификатору можно запустить сбор клиентов по ней. Используется в сервисе маршрутов. При сохранение методом **SaveSelection** такая выборка сохранится и для запуска необходимо вызвать метод **StartSelection**
 - Одноразовая выборка - запускается один раз при сохранение.
 - коллбэк для ошибок - метод для обработки, если выборка не отработала либо прервалась в процессе. Сигнатура (??)
- Условия выборов хранятся на сервисе выборов. Для маршрутов предоставляется целочисленный идентификатор сущности, по этому идентификатору методом **StartSelection** запускается отбор клиентов и поставка их в callback.

Так же разрабатывается новый метод проверки вхождения вектора сущностей в выборку по сигнатуре:

- EntityIds[] - вектор проверяемых идентификаторов
- TaskID - идентификатор выборки

На выходе:

- Рекордсет из идентификаторов подошедших под условия выборки

Модель данных

Таблица Route

Поле	Тип	Описание
StartType	Int	Тип запуска маршрута 0 - разовый 1 - по событию 2 - по таймеру

Модель маршрута

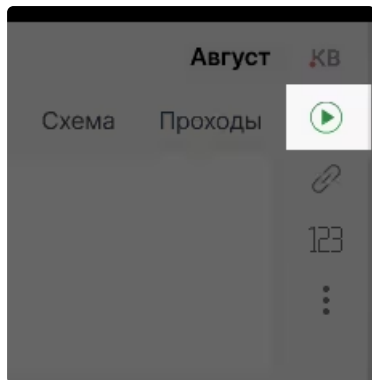
Поле	Тип	Есть в списочном методе	Описание
StartMode	Record	✓	Тип запуска маршрута 0 - разовый 1 - по событию 2 - по таймеру

Модель запуска (StartMode)

Модель запуска (StartMode)

Поле	Тип	Описание
Type	Int	Тип запуска маршрута 0 - разовый 1 - по событию 2 - по таймеру
Filter	Int	Идентификатор выборки
FilterType	Int	0 - Клиенты
Trigger	UUID	Идентификатор триггера для запуска
PeriodicityData	Record	Параметры создания планового события

API запуска маршрута



В интерфейсе запуска сценария осуществляется через кнопку запуска. Для управления маршрутом предоставляются новые методы API:

- **Route.Start(RouteUUID)** - для запуска сценария. Внутри метод получает информацию о маршруте, проверяет текущий статус маршрута и в зависимости от сценария:
 - Разовый запуск - вызов метода **Route.StartTrace(RouteUUID, Params)**
 - По событию - вызов метод **Trigger.Create** - обновив состояние, что триггер запущен.
 - По расписанию - вызов в C++ коде метода **AddTask (API)** для постановки планового события
- **Route.Stop(RouteUUID)** - для остановки сценария. Внутри метод получает информацию о маршруте, проверяет текущий статус маршрута и в зависимости от сценария:
 - Разовый запуск - остановка выборки методом **CoreV3.Cancel/1**
 - По событию - вызов метод **Trigger.Delete** - обновив состояние, что триггер выключен.
 - По расписанию - вызов в C++ коде метода **RemoveTask (API)** для остановки планового события
- **Route.Status(RouteUUID)** - для статуса сценария. Для получения статуса выборки **CoreV3.ReadSelection/1** в поле **ClientCount** будет кол-во найденных данных. Сценарий может находиться в состояниях:
 - draft - настроен, но не запущен.
 - starting - маршрут запускается.
 - running - маршрут активен.
 - stopping - идет остановка проходов.
 - stopped - маршрут остановлен.
 - error - ошибка при запуске или выполнение маршрута.

Для постановки маршрута на авто-прохождение добавляется метод:

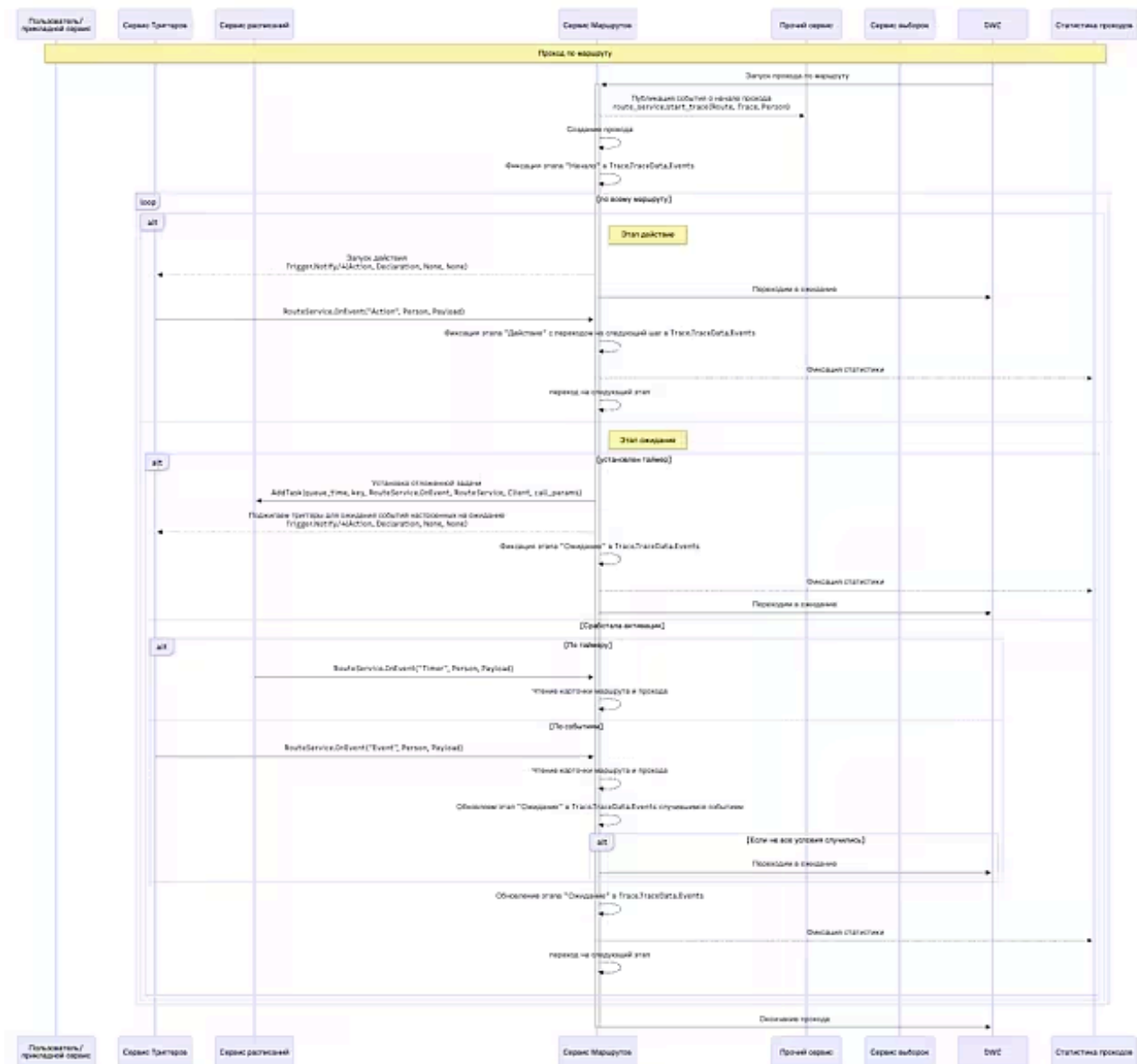
- **Route.StartTrace(RouteUUID, Params)** - получает информацию о маршруте, переменные пред фильтрации клиента и принимает решение о запуске через выборку клиентов или напрямую задачу в DWC, если передан конкретный список клиентов в Params и не настроена выборка. Ограничения метода:
 - Если не переданы клиенты либо не настроена выборка - авто-проходчик не может быть запущен.
 - При настроенной переменной окружения клиента **Category.silence_mode = True** запуск маршрутов по категории не производится.

6.0.2.2 Авто-проходчик маршрутов

<https://online.sbis.ru/knowledge-bases/f6118557-9c5f-4480-8276-499205f995c8?folder=3e5f52f2-255b-4930-b3b4-17d4923a2cad&published=true&mode=readList>

Проход - это фиксация прохождения пользователем (роботом) по выбранному маршруту. В проходе фиксируются: пользователь, маршрут, время начала и окончания, время попадания и ухода с каждого этапа, дополнительные данные, введенные пользователем.

Переход - факт перемещения пользователем по маршруту от этапа куда-либо в другое место (другой этап, завершение, переход во внешний маршрут, и т.д.)



▼ Схема

```

1 ---
2 config:
3   theme: default
4   look: neo
5   fontFamily: "'Recursive Variable', sans-serif"
6   themeVariables:
7     fontFamily: "'Recursive Variable', sans-serif"
8 ---
9 sequenceDiagram
10
11 participant A as Пользователь/прикладной сервис
12 participant B as Сервис Триггеров
13 participant C as Сервис расписаний
14
15 participant D as Сервис Маршрутов
16 participant E as Прочий сервис
17 participant H as Сервис выборки
18 participant F as DW
19 participant G as Статистика процессов
  
```

```

20
21     Note over A,G: Проход по маршруту
22     F -->+ D: Запуск прохода по маршруту
23     D --> E: Публикация события о начале прохода
24     <br>route_service.start_trace(Route, Trace, Person)
25
26     D --> D: Создание прохода
27     D --> D: Фиксация этапа "Начало" в Trace.TraceData.Events
28     loop по всему маршруту
29         alt
30             Note right of D: Этап действие
31             D --> B: Запуск действия <br>Trigger.Notify/4(Action,
32             Declaration, None, None)
33             D --> F: Переходим в ожидание
34             B --> D: RouteService.OnEvent("Action", Person, Payload)
35             D --> D: Фиксация этапа "Действие" с переходом на
36             следующий шаг в Trace.TraceData.Events
37             D --> G: Фиксация статистики
38             D --> D: переход на следующий этап
39
40         else
41             Note right of D: Этап ожидание
42             alt установлен таймер
43                 D --> C: Установка отложенной задачи <br>
44                 AddTask(queue_time, key, RouteService.OnEvent, RouteService, Client,
45                 call_params)
46                 D --> B: Поджигаем триггеры для ожидания событий
47                 настроенных на ожидание <br>Trigger.Notify/4(Action, Declaration,
48                 None, None)
49                 D --> D: Фиксация этапа "Ожидание" в
50                 Trace.TraceData.Events
51                 D --> G: Фиксация статистики
52                 D --> F: Переходим в ожидание
53                 else Сработала активация
54
55                 alt По таймеру
56                     C --> D: RouteService.OnEvent("Timer", Person,
57                     Payload)
58                     D --> D: Чтение карточки маршрута и прохода
59                 else По событиям
60                     B --> D: RouteService.OnEvent("Event", Person,
61                     Payload)
62                     D --> D: Чтение карточки маршрута и прохода
63                     D --> D: Обновляем этап "Ожидание" в
64                     Trace.TraceData.Events случившимся событием
65                     alt Если не все условия случились
66                         D --> F: Переходим в ожидание
67                     end
68                 end
69                 D --> D: Обновление этапа "Ожидание" в
70                 Trace.TraceData.Events
71                 D --> G: Фиксация статистики
72                 D --> D: переход на следующий этап

```

```

61         end
62     end
63
64 end

```

Авто-проходчик - это серверный робот, который движет клиента по маршруту автоматически. Он последовательно выполняет действия на каждом этапе и сам вычисляет, на какой следующий этап перейти, опираясь на условия переходов, заданные на рёбрах маршрута.

На каждом этапе маршрута авто-проходчик:

1. **Выполняет действие** - список действий, заданных на узле (действия, ожидания). Это вызовы запуск триггерных действий (например, «начислить бонусы», «отправить рассылку», «записать результат»).
2. **Фиксирует переход** - Каждый этап маршрута записывает событие в **Trace.TraceData.Events**: с какого узла, по какому ребру, время, достигнутые цели, результат и т.д.
3. **Завершает проход** - если узел имеет тип 2 (точка выхода), фиксирует завершение прохода.
4. **Публикация события о движении этапа** (after,before) - для исполнения прикладной логики



Авто-проходчик **не работает постоянно**. Он запускается по одному из трёх внешних триггеров (ручной, событие, расписание) и прогоняет проход до упора - до блока-ожидания или до завершения маршрута

Запуск авто-проходчика

<https://online.sbis.ru/page/knowledge-bases/694fcc5c-abd7-4c2b-9c9b-761e1fdc92fb?mode=readList&folder=83b2e36a-44d8-42d6-861d-4b7848c9225a&published=true>

Вызывающая сторона обращается к сервису маршрутов через публичный метод **RouteService.StartTrace**. Внутри метода формируется DWC-сценарий, по одной задаче на клиента и отправляет его в **DWC**. DWC вызывает метод запуска автопрохода **RouteService.StartClientRoute** на сервисе маршрутов для каждой задачи.

Логика прохождения этапов

Каждый переход авто-проходчика записывается как событие в Events

Поле	Значение
Uuid	Уникальный идентификатор события
Begin	Момент входа на узел
End	Момент выхода (после вычисления перехода)
Node	UUID текущего узла
Pass	UUID выбранного ребра
Targets	Достигнутые цели (если ребро/узел помечены как цель)
Result	0 — нейтральный, 1 — положительный, 2 — отрицательный
StepNumber	Инкрементальный порядковый номер шага

Эти данные уходят в **TraceCounter** и становятся основой для статистики переходов, которая уже строится существующими средствами платформы.

Действие

Описано в ТЗ в блоке "Этап действие -> Модель данных -> Модель события"

Условие

Поле	Тип	Значение
Id	Uuid	Uuid()
Begin	DateTime	Время входа на узел

End	DateTime	Время после вычисления условий
Node	Uuid	UUID узла
Pass	Uuid	UUID выбранного ребра
Result	Int	Из конфигурации узла: 0 / 1 / 2
Targets	Uuid[]	Цели узла
StepNumber	Int	N
Error	Bool	false

Ожидание

Описано в ТЗ в блоке "Этап ожидание -> Модель данных -> Модель события"

Завершение

Авто-проходчик дошёл до узла **type=8** или не нашёл ни одного доступного ребра. После записи в проходе записывается информация о завершение прохода **Trace.IsFinished = true, Trace.End = now()**

Заполнение модели events:

Поле	Тип	Значение
Id	Uuid	Uuid()
Begin	DateTime	Время входа на узел
End	DateTime	Время завершения
Node	Uuid	UUID узла
Pass	Uuid	Null - финальный узел
Result	Int	Из конфигурации узла: 0 / 1 / 2
Targets	Uuid[]	Цели узла
StepNumber	Int	N
Error	Bool	false

Исполнение этапа «Действие»

Модель узла

Поле	Тип	Описание
Action	Uuid	UUID действия. Передается в поле Data.TriggerID

Исполнение этапа

При входе в

- При входе : вызывается **Trigger.Notify/4**(Action, Declaration, None, None).
- Обработка результата: После успешного запуска действия сервис маршрутов получает событие через **RouteService.OnEvent("Action", Person, Payload)**.
- После успешного запуска действия фиксируется этап «Действие» в **Trace.TraceData.Events** с переходом на следующий шаг маршрута.
- Фиксация статистики прохода для аналитики.

Исполнение этапа «Ожидание»

Этап "Ожидание" - это специальный тип узла маршрута, который приостанавливает движение по маршруту на заданное время (через сервис расписания ([task-scheduler](#)) (согласно [API](#))) ожидая либо истечения таймера, либо наступления заранее настроенного события.

События выбираются из справочника доступных для выбора событий, сгруппированных по иерархии объектов.

На одном блоке можно настроить **любое количество событий**. Каждое событие:

- имеет свое название
- ведёт на своё отдельное ребро выхода из блока

У блока всегда минимум **одно ребро** (выход по таймеру) и **ноль или более рёбер** по событиям.

Модель узла

Поле	Тип	Описание
Timeout	Int	Сколько дней/часов ждать до истечения таймера. null - таймер не устанавливается
TimeoutPassId	Uuid	UUID ребра выхода при истечении таймера.
WaitEvents	Array[Json]	Список событий для выхода
WaitEvents[].EventType	String	Идентификатор типа события из справочника
WaitEvents[].PassId	Uuid	UUID ребра выхода при срабатывании события.
WaitPassId	Uuid	Общее ребро выхода при выполнении всех условий.

Исполнение этапа

Заход в этап

- При наличии настроенного таймера выполняется установка отложенной задачи в СПС:
AddTask(queue_time, key, RouteService.OnEvent, RouteService, Client, call_params).
- Фиксируется этап «Ожидание» в **Trace.TraceData.Events** и проход переходит в состояние ожидания.
- Фиксация статистики прохода для аналитики.

Срабатывание таймера

- По наступлению времени таймера вызывается **RouteService.OnEvent("Timer", Person, Payload)**
- Продолжает проход по маршруту на следующий этап

Событие

- При внешнем событии вызывается **RouteService.OnEvent("Action", Person, Payload)**
- Обработчик считывает карточку маршрута и карточку прохода, обновляет этап «Ожидание» в **Trace.TraceData.Events** случившимся событием
- Далее либо снова переходит в ожидание (если ещё не все условия выполнены), либо продолжает проход по маршруту на следующий этап

Подпроходы

Подпроход - совокупность всех переходов, выполненных в рамках одного конкретного экземпляра маршрута.

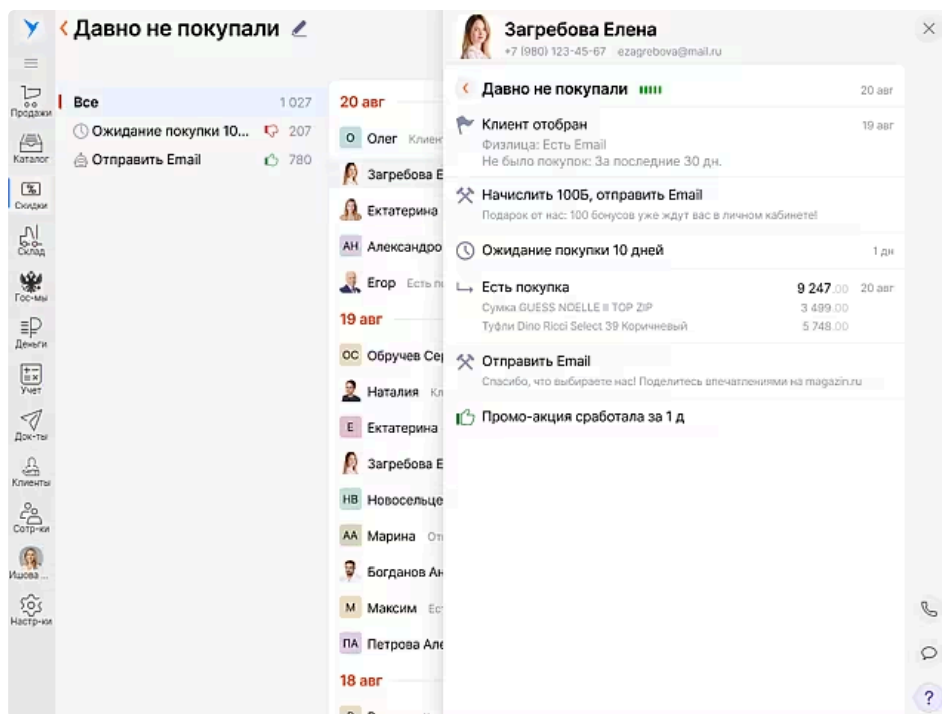
Подпроход может быть остановлен при выходе в модуль или подпрограмму, а так же возобновлён, при возврате из модуля или подпрограмме по точке выхода.

Исполнение этапа

Для сложных маршрутов предусмотрен «Этап подпроход», когда внутри основного прохода запускается вложенный проход по подпроцессу. В таком случае запускается проход по подмаршруту.

6.0.2.3 Карточка прохода

Карточка прохода будет реализована отдельной [pagex страницей](#), будет настроена предзагрузка данных согласно платформенной [документации](#). Декомпозию панели см в [разделе](#).



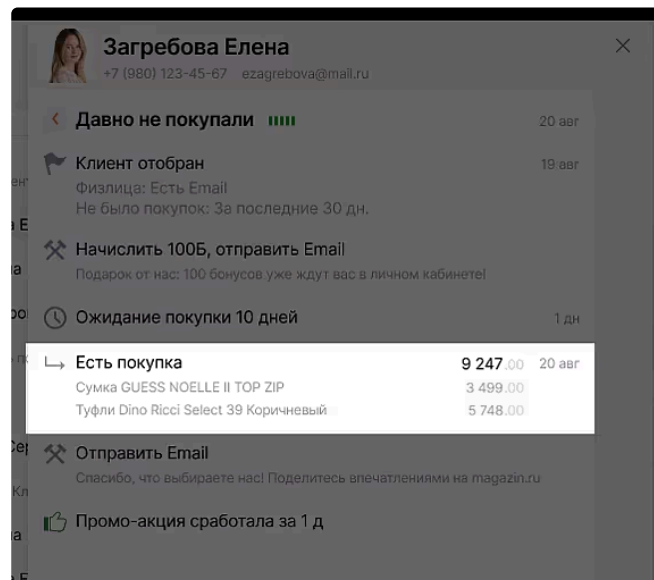
В качестве источника данных для страницы будем использовать нашу БЛ обертку над методом платформы маршрутов - [RouteService.FindTrace](#).

Согласно документации, метод вернет нам как список пройденных этапов конкретного прохода, так и список всех проходов по маршрутам конкретного клиента.

Список этапов прохода метод предоставляет в виде списка [Events](#), содержащих только UUID этапов и переходов (Node и Pass соответственно). Для формирования человекочитаемых строк в интерфейсе клиентской частью будет выполнено сращение событий из FindTrace с моделью маршрута, полученной при открытии карточки маршрута по формату `RouteService.Read`.

Метаданные о прохождении этапа (Дата попадания и ухода с этапа, данные продажи, возможные ошибки при выполнении действий и пр) так же будут доступны в списке Events.

6.0.2.3 Детализация о продаже в карточке прохода



Во время авто-прохода прикладное событие может вернуть данные о метриках, которые необходимо зафиксировать на событие, а в дальнейшем агрегировать на маршрут.

К примеру, чтобы ответить на вопрос, сколько мы заработали с продаж, напомнив о брошенной корзине.

Данные события приходят в формате списка:

```

1  {
2    "metrics": [
3      {

```

```

4      "Type": "purchase",    // Тип события
5      "Key": "abc-123",      // Ключ события
6      "Amount": 1500.00,     // Метрика
7      "Currency": "RUB",     // Валюта
8      "OccurredAt": "2026-04-14T10:00:00Z", // Время операции
9      "OptionalData": {}     // Прикладные данные для рендера
10     }, ...
11 ]

```

Модель метрик события (Metrics)

При триггерном событии авто-проходчик записывает данные о транзакции в прикладное поле `Trace.TraceData.Events.Metrics`

Поле	Тип	Описание
Type	String	Тип события (Продажа, Бронирование и т.д)
Key	String	Ключ события
Amount	Double	Числовая метрика
Currency	String	Код валюты
OccurredAt	Datetime	Дата и время события
OptionalData	JSON	Прикладные поля

6.0.2.3 Глобальная пред фильтрация Клиентов

Для настройки запускаемых категорий в предметной области на конкретном клиенте вводим работу через сервис глобальных переменных.

Платформа маршрутов вводит параметры:

- **silence_mode**(bool) - режим тишины. Если переменная = True, значит новые маршруты не запускаются. Прерывание происходит в методе **Route.StartTrace/2**.
- **one_user_run_pass** (bool) - одновременно может быть запущен маршрут только 1 раз для лица. Проверка происходит в 2 точках:
 - В клиентском модуле **Route.StartTrace/2** - если в параметрах указан конкретный клиент и по нему запущен маршрут.
 - На сервисе маршрутов в **RouteService.StartTrace/3** - если клиент получен из выборки для запуска авто-прохода.
- **no_more_often_term** (record) - запускать не чаще N дней/часов. Проверяется на сервисе маршрутов в **RouteService.StartTrace/3**. Задается полями:
 - **term** - целое число
 - **type** - 0 - дни, 1 - часы.

Пример: `Loyalty.silence_mode = true`

6.1 Архитектура данных и связь понятий

Сценарий-Маршрут (Лояльности) - Является **Маршрутом**. Задаёт периодичность и правила отбора попадания клиента на маршрут, схему прохождения клиентов и перечень целей

- Проход** - Совокупность переходов клиента от попадания на маршрута до завершения

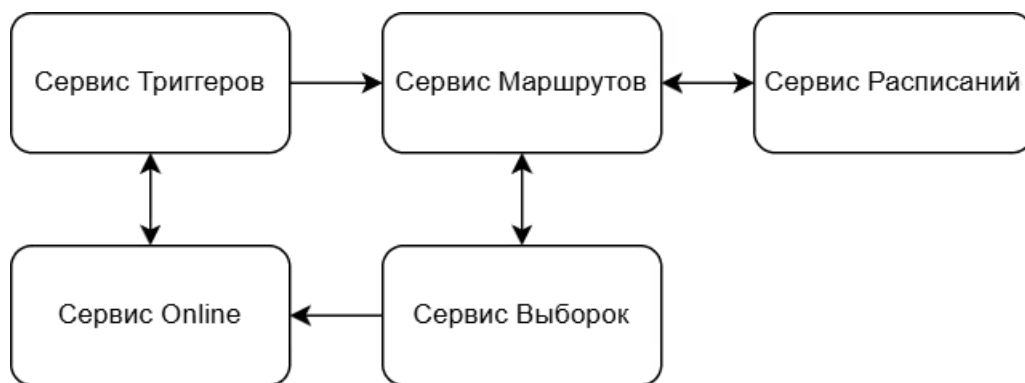
- Покупка** - Осуществленная покупка клиентом находящимся на маршруте в момент не завершенного прохода

- **Событие** - Какое либо событие определяемое прикладной областью (например "совершена покупка")
- **Действие** - Какое либо действие определяемое прикладной областью как реакция на событие задающееся в Триггере
- **Объект** - Определяет множество триггерных Событий и множество возможных на них Действий

На маршруте этапа "Действие" будет соответствовать тех. триггер (создаваемый платформой маршрутов) с заполненным блоком действия, но не с заполненным блоком событие т.к. данный триггер будет "поджигаться" платформой маршрутов в авто-проходчике



Общая схема взаимодействий сервисов в сценариях прохождения Клиента по маршруту. Для асинхронного взаимодействия с сервисами используется DWG.



- Сервис Online -> Сервис Триггеров - Генерация триггерного события
- Сервис Триггеров -> Сервис Online - Выполнение действия определенного в триггере
- Сервис Выборок -> Сервис Online - Отбор Клиентов
- Сервис Выборок -> Сервис Маршрутов - Отправка множества отображенных Клиентов
- Сервис Маршрутов -> Сервис Расписаний - Постановка таймера
- Сервис Расписаний -> Сервис Маршрутов - Вызов callback поставленного таймера
- Сервис Триггеров -> Сервис Маршрутов - Инициализация обработки добавления клиента на маршрут или просыпание с этапа "Ожидание"

Прочие межсервисные взаимодействия в разрезе других операций см. в [ТЗ 6.0.2.1 Помещение клиента на маршрут](#), [ТЗ 6.0.2.2 Авто-проходчик маршрутов](#)

6.3 Описание API

6.3.1 Используемые API

[Публичное API маршрутов](#)

[API сервиса расписаний](#)

6.3.2 Новые API

RouteService.StartTrace - вызывается прикладными разработчиками для запуска авто-прохода по маршруту.

Входные параметры:

Параметр	Тип	Обязательный	Описание
Route	uuid	Да	Идентификатор маршрута
Client	int64	Да	Идентификатор клиента
Persons	int64[]	Да	Список идентификаторов персон для которых будет запущен маршрут
Params	Record	Да	Параметры запуска маршрута

Доступные параметры в Params:

Параметр	Тип	Обязательный	Описание
Version	string	Нет	Версия
Redaction	Uuid	Нет	Идентификатор редакции
Reason	int	Да	Причина запуска: 0 - manual - ручной запуск 1 - event - по событию 2 - planed - по расписанию

EventType	String	Нет	Тип события, инициировавшего запуск (например loyalty.purchase). Передаётся при StartReason=event

RouteService.OnEvent - вызывается триггером при наступлении прикладного события. Внутри метод запускает авто-обходчик, чтобы он продолжил движение по маршрутам, для которых событие предназначено.

Параметр	Тип	Обязательный	Описание
Route	uuid	Да	Идентификатор маршрута
Client	int64	Да	Идентификатор клиента
Payload	json	Нет	Прикладные данные о транзакции

RouteService.OnEvent - вызывается сервисом расписаний после завершения таймера. Внутри метод запускает авто-обходчик, чтобы он продолжил движение по маршруту.

Параметр	Тип	Обязательный	Описание
Route	uuid	Да	Идентификатор маршрута
Client	int64	Да	Идентификатор клиента

6.4 Схема хранения данных

- Данные сценария лояльности, ложатся на модель хранения Маршрута и не предполагают какого либо расширения
- Факт покупки Клиентом в разрезе маршрута, хранится на проходе см. подробнее [ТЗ 6.0.2.3 Детализация о продаже в карточке прохода](#) без возможности агрегации по сумме покупок (но возможность агрегации закладывается на будущее)

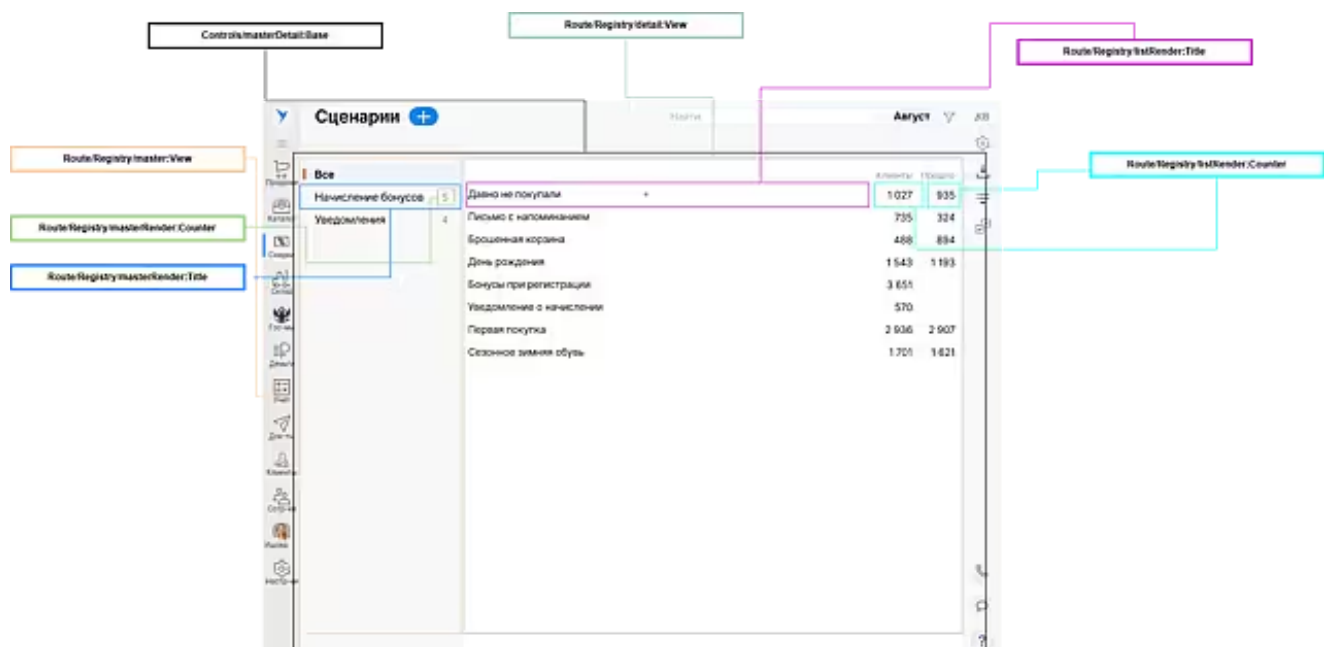
6.4.1 Описание основных выборок

Без изменений.

6.5 Пользовательский интерфейс

6.5.1 Декомпозиция пользовательского интерфейса

Реестр сценариев



Для настройки мастера необходимо передать в массив колонок **Route/Registry/master:View** компонент **Title** с свойствами:

- titleProperty
- counter

▼ Пример кода

```
1 <Title
2   titleProperty="title"
3   counter={<Counter counterProps="count" />}
4 />
```

💡 **title** - поле списка для вывода названия
count - поле для вывода счетчика

Для настройки деталей необходимо передать в массив колонок **Route/Registry/detail:View** компоненты **Title** и **Counter** с соответствующими свойствами.

▼ Пример кода

```
1 const columns: IColumnConfig[] = React.useMemo(() => {
2   return [
3     {
4       key: 'title',
5       render: (
6         <Title
7           titleProperty="title"
8         />
9       ),
10    },
11    {
12      key: 'countClients',
13      render: (
14        <Counter
15          counterProperty="count"
16        />
17      ),
18      width: 'max-content',
19      getCellProps: (_item: Model) => ({
```

```

20             halign: 'right',
21         })),
22     },
23     {
24         key: 'count',
25         render: (
26             <Counter
27                 counterProperty="count"
28             />
29         ),
30         width: 'max-content',
31         getCellProps: (_item: Model) => ({
32             halign: 'right',
33         }),
34     },
35 ] as IColumnConfig[];
36 }

[1]):

```



title - поле списка для вывода названия

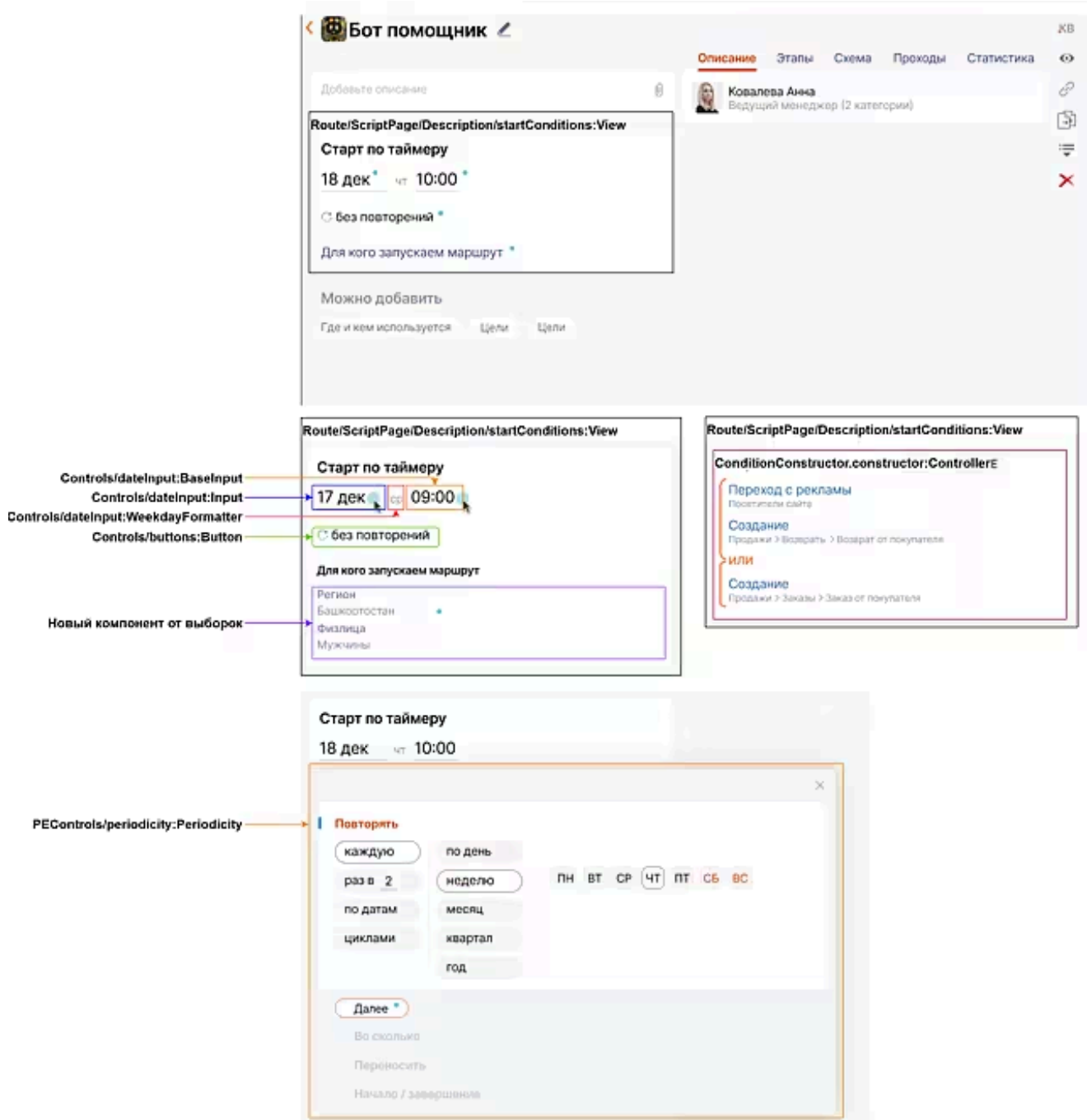
count - поле для вывода счетчика

Вкладка Описание

Отображение условия отбора и типа сценария должны быть одним компонентом. На данный момент в макете текущего проекта они разделены.

Декомпозиция пользовательского интерфейса таймера:

https://project.sbis.ru/uuid/fe24fac5-a4e0-4f0c-9bff-c87f3b8c2679/page/project-main#toc_1398ce8f-9a22-481c-80db-dc4ebc3e7dbc



В компонент страницы нужно будет встроить `Route/ScriptPage/Description/startConditions:View`.

Вкладка Этапы


```

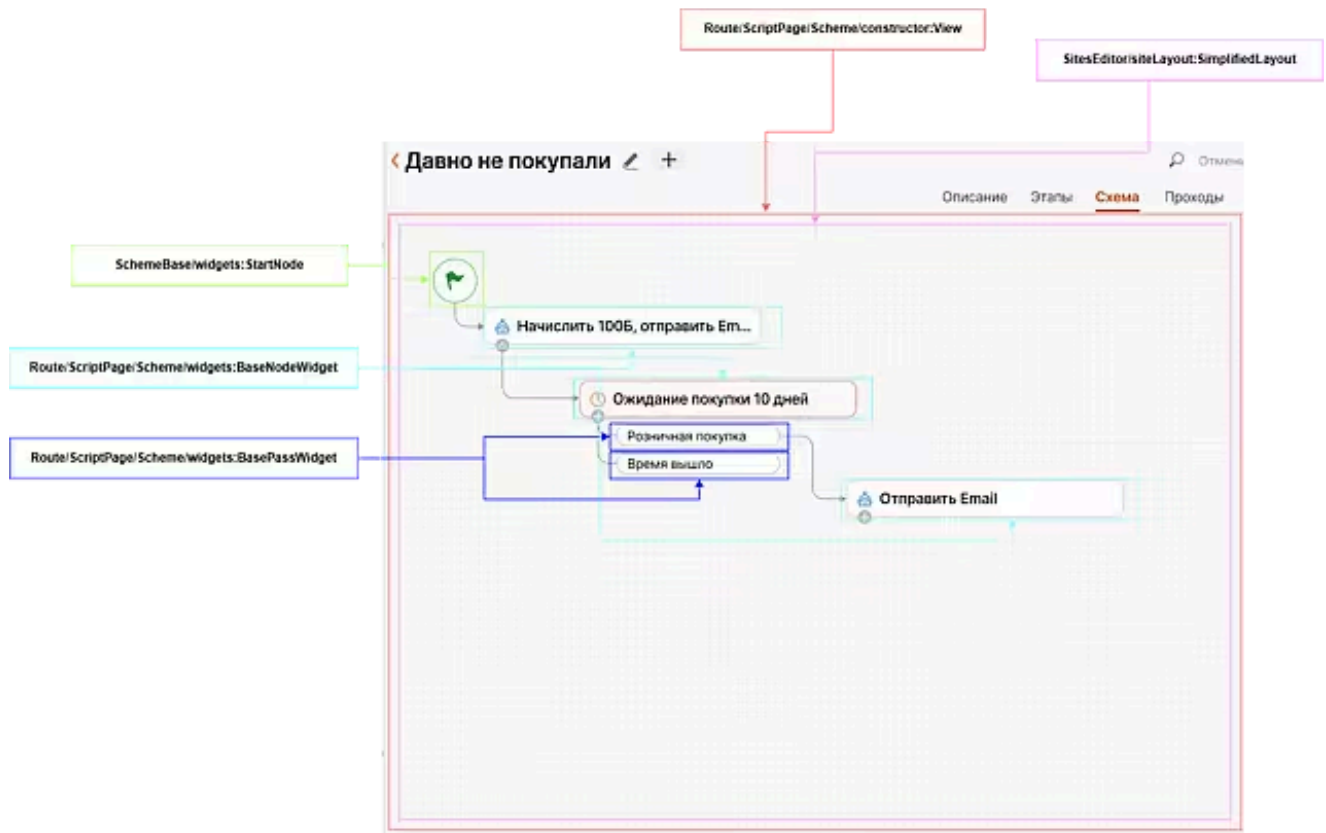
▼ Passes: {
  ▼ rs: [
    ▼ 0: {
      Id: null,
      Uuid: "c565da97-2ba3-4bb0-ad9c-0838bdf1dcd6",
      ▼ NodesTo: [
        0: "01c4f54b-8da8-4edd-acc9-08a192b0fc68"
      ]
      Name: "Один на основании другого",
      Type: 0,
      Result: null,
      Order: null,
      Actions: null,
      Conditions: null,
      Targets: null,
      IsFolder: null,
      Folder: null,
      IsVisible: null,
      Data: "{"d":["c565da97-2ba3-4bb0-ad9c-0838bdf1dcd6","379899f"],"true",null,[],[],["d":[","s":[{"n":"i","a"},{"n":"scriptId","t":"Строка"},{"n":"stepId","a"}],"_type":"recordset"],["01c4f54b-8da8-4edd-acc9-08a192b0fc68"],null,null,0,null,0],"s":[{"n":"id","t":"UUID"},{"n":"parent","t":"{"n":"Массив","t":"UUID"},{"n":"isSubItem","t":"{"n":"логическое"},{"n":"answers","t":"{"n":"Массив","t":"{"n":"Массив","t":"Текст"},{"n":"targets"},{"n":"conditions","t":"Выборка"},{"n":"childStep","t":"Строка"},{"n":"folder","t":"UUID"},{"n":"condition","t":"{"n":"Массив","t":"Текст"},{"n":"blockType","t":"Число целое"},{"n":"colorName","t":"Число целое"}],"_type":"record"}}",
      Mark: "success",
      _IsNewMapping: true,
      _OldType: 0
    }
  ]
  ▼ 1: {
    Id: null.
  }
}

```

Модель перехода:

<https://wi.sbis.ru/page/autodoc-ts/Route/Models/Scheme/base/PassModel?v=26.3200>

Вкладка Схема



Компонент **Route/ScriptPage/Scheme/constructor:View** необходим для отображения страницы схемы. Каждый элемент схемы является виджетом. Есть платформенный набор виджетов схемы и базовый набор **Route** (базовые обёртки для первого набора).

Подробный список:

[SchemeBase widgets](#)

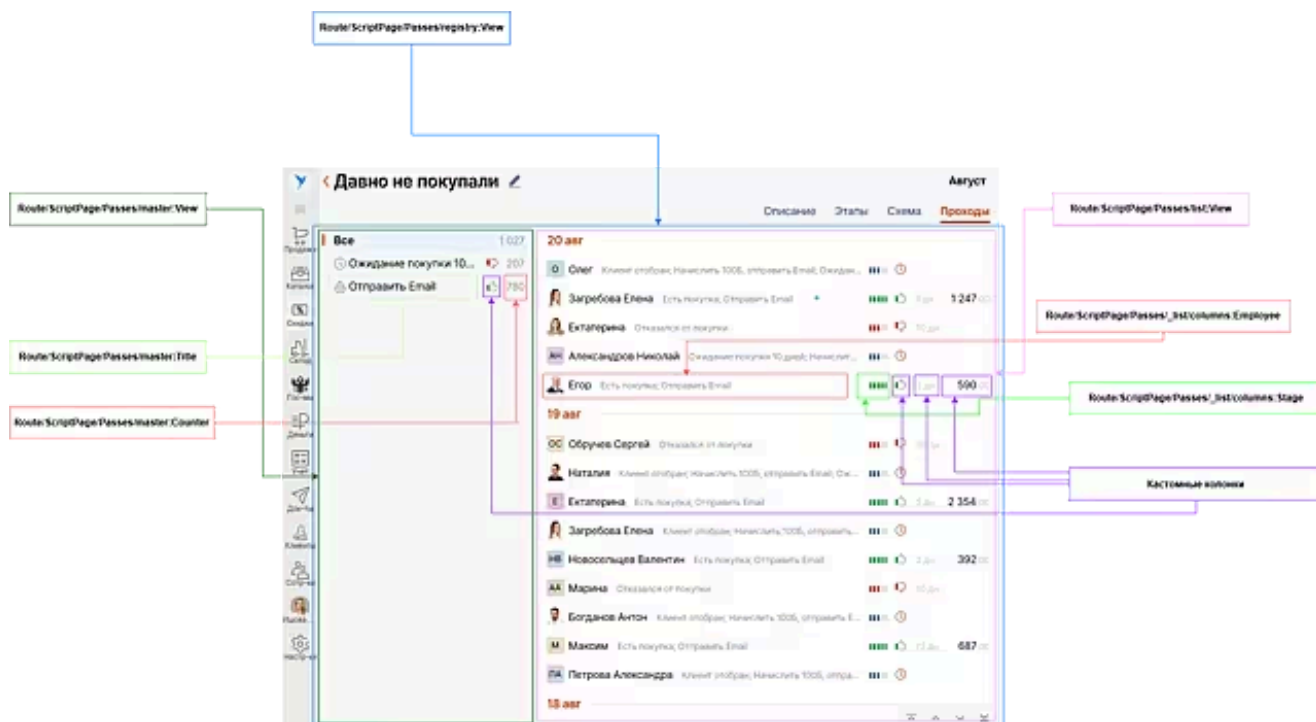
[Route widgets](#)

Обязательное условие - использование общей модели маршрута **SchemeElementModel**.

<https://wi.sbis.ru/page/autodoc-ts/Route/Models/Scheme/base/SchemeElementModel?v=26.3200>

За отображение виджета отвечает поле модели **EditorType**, возвращающее путь до элемента схемы.

Вкладка Проходы



Для встраивания страницы необходимо использовать готовый компонент **Route/ScriptPage/Passes/registry:View** и передать основные опции

- **viewMode** (в данном случае - 'custom')
- **masterFilters** (формирование списка мастера)
- **masterStoreIds** - (идентификаторы контекстов)

и другие. Подробности

<https://wi.sbis.ru/page/autodoc-ts/Route/ScriptPage/Passes/registry/IView?v=26.3100>

▼ Примеры реализации

[https://git.sbis.ru/reclamations/contract-service/-/blob/rc-26.4200/client/Chatbot/Pages/_passes/View.tsx?](https://git.sbis.ru/reclamations/contract-service/-/blob/rc-26.4200/client/Chatbot/Pages/_passes/View.tsx?ref_type=heads)

[ref_type=headshttps://git.sbis.ru/telephony/sale-scripts/-/blob/rc-26.4100/client/SaleScripts/Pages/_passes/View.tsx?ref_type=heads](https://git.sbis.ru/telephony/sale-scripts/-/blob/rc-26.4100/client/SaleScripts/Pages/_passes/View.tsx?ref_type=heads)

При формировании колонок мастера необходимо использовать следующие компоненты:

- **Route/ScriptPage/Passes/master:Title** (необходимо добавить с возможностью обогащением иконкой);
- **Route/ScriptPage/Passes/master:Counter**.

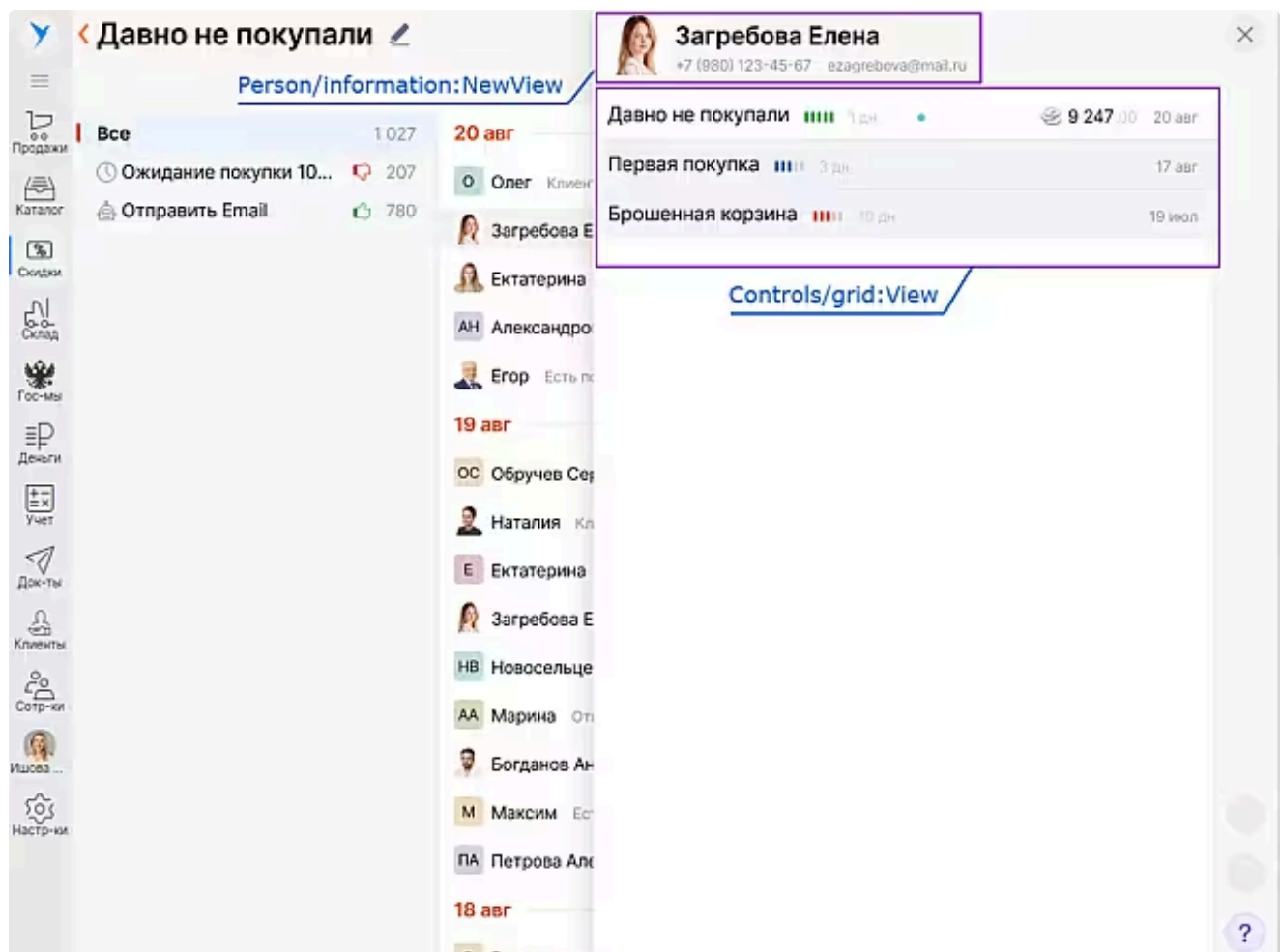
При формировании колонок детейла необходимо использовать следующие компоненты:

- **Route/ScriptPage/Passes/_list/columns:Employee** (необходима доработка - возможность добавить прикладной текст после сотрудника);
- **Route/ScriptPage/Passes/_list/columns:Stage**. (необходима доработка - добавить зеленый цвет в покраску).

Конфигурацию колонки списка детейла можно получить при помощи вызова соответствующего хука из

<https://wi.sbis.ru/page/autodoc-ts/Route/ScriptPage/Passes/list?v=26.3100>

Карточка прохода



Декомпозиция карточки Прохода

